

DOKUMENTASI TEKNIS SIMASTER

Sistem Manajemen Aset
General Affair
2026

Panduan Tim IT

LM FEB UI



DAFTAR ISI

PENDAHULUAN	5
1.1 Tentang Dokumen Ini	5
1.2 Arti Kotak Penjelasan.....	5
1.3 Bagian Mana yang Perlu Dibaca.....	5
1.4 Tentang SIMASTER.....	6
1.5 Siapa yang Menggunakan Aplikasi Ini	7
1.6 Cara Kerja Sistem dalam Satu Gambar	7
1.7 Pola Umum Setiap Pengajuan	8
1.8 Daftar Modul (Fitur) Utama	8
1.9 Hal Penting yang Perlu Diketahui Sejak Awal	9
TEKNOLOGI YANG DIPAKAI.....	10
2.1 Bahasa & Framework	10
2.2 Library PHP (Production)	10
2.3 Library PHP (Development)	10
2.4 Library Frontend.....	11
2.5 Database & Storage.....	11
2.6 Infrastruktur & Deploy	12
2.7 Kebutuhan Server	12
2.8 Tools Build, Test & Format	15
2.9 Layanan Pihak Ketiga	15
ARSITEKTUR: BAGAIMANA SISTEM DISUSUN	16
3.1 Diagram Arsitektur (Production)	16
3.2 Komponen & Fungsinya.....	17
3.3 Alur Request HTTP.....	18
3.4 Struktur Folder	19
3.5 Pola Desain.....	20
3.6 Modul-Modul Utama.....	22
ALUR KERJA SETIAP MODUL.....	23
4.1 Pola Umum Modul Transaksional	23
4.2 Peminjaman Aset	24
4.3 Peminjaman Ruangan	25

4.4	Permintaan Barang Gudang	26
4.5	Pengadaan Barang/Jasa (Pembelian ke Vendor)	27
4.6	Permintaan Kendaraan	27
4.7	Pemindahan Aset.....	28
4.8	Ekspedisi (Pengiriman Keluar).....	28
4.9	Pengaduan Kerusakan	29
4.10	Maintenance	30
4.11	Stok Opname Gudang	31
4.12	Transaksi Gudang (Masuk / Keluar).....	31
4.13	Laporan Pemusnahan Aset.....	32
4.14	Rekap Bulanan Gudang (Scheduled Task)	32
4.15	Anggaran RKAT	33
4.16	Form Publik — Keamanan	34
ALUR DATA & STRUKTUR DATABASE		35
5.1	ERD per Modul	35
5.2	Daftar Endpoint per Modul	40
5.3	Format ID (Primary Key)	43
5.4	Data Flow: Realisasi RKAT Otomatis.....	43
5.5	Audit Log — Semua Perubahan Data.....	44
INSTALASI, DEPLOYMENT & BACKUP.....		45
6.1	Setup Lokal (Development).....	45
6.2	Environment Variables (.env)	46
6.3	Docker (Development)	47
6.4	Docker (Production)	48
6.5	Cloudflare Zero Trust Tunnel.....	49
6.6	Backup & Restore	49
6.7	Migration Policy.....	51
PANDUAN TROUBLESHOOTING		52
7.1	Arti Pesan Error yang Sering Muncul	52
7.2	Lokasi Log.....	52
7.3	Masalah Umum & Solusi.....	53
7.4	Checklist Diagnosa Cepat.....	58

7.5	Kapan Harus Eskalasi?.....	59
KEPUTUSAN STANDAR & DAFTAR RISIKO		61
8.1	Bagian A — Keputusan Standar.....	61
8.2	Bagian B — Risk Register.....	63
8.3	Ringkasan Prioritas	65
SOP OPERASIONAL HARIAN.....		66
9.1	SOP-01: Tambah User Baru	66
9.2	SOP-02: Nonaktifkan / Hapus User	66
9.3	SOP-03: Reset Password User.....	67
9.4	SOP-04: Buat Role Baru	67
9.5	SOP-05: Set Akses Menu per Role (RBAC).....	68
9.6	SOP-06: Tambah Menu Baru ke Sistem	68
9.7	SOP-07: Clear Cache (Setelah Deploy / Konfigurasi Berubah)	69
9.8	SOP-08: Restart Queue Worker	69
9.9	SOP-09: Jalankan Rekap Bulanan Gudang Manual	70
9.10	SOP-10: Export Data ke Excel/PDF.....	70
9.11	SOP-11: Backup Manual Database.....	71
9.12	SOP-12: Proses Darurat — Database Tiba-tiba Tidak Bisa Diakses (Production)	71
9.13	Referensi Cepat.....	72
LAMPIRAN: GLOSARIUM ISTILAH		73

PENDAHULUAN

1.1 Tentang Dokumen Ini

Dokumen ini adalah panduan teknis SIMASTER (Sistem Manajemen Aset) LM FEB UI untuk tim IT: developer, DevOps/infrastruktur, dan support. Isinya menjelaskan teknologi yang dipakai, susunan sistem, alur kerja setiap modul, alur data, cara instalasi dan backup, panduan troubleshooting, daftar risiko, serta SOP operasional harian.

Dokumen ini ditulis untuk dua jenis pembaca sekaligus: orang teknis yang perlu detail (perintah, nama file, struktur database) dan orang non-teknis yang cukup memahami gambaran besarnya. Karena itu, setiap bab dilengkapi kotak penjelasan berbahasa sederhana. Untuk panduan pemakaian aplikasi sehari-hari (klik menu, isi formulir), gunakan Manual Pengguna SIMASTER.

1.2 Arti Kotak Penjelasan

SINGKATNYA

Kotak "SINGKATNYA" ada di awal setiap bab: ringkasan isi bab dalam bahasa sehari-hari. Jika hanya punya waktu sedikit, cukup baca kotak ini.

Penjelasan sederhana

Kotak "Penjelasan sederhana" atau "Dalam bahasa sederhana" menerjemahkan bagian teknis menjadi penjelasan yang mudah dipahami.

Perhatian

Kotak kuning "Perhatian" menandai hal penting atau berisiko yang harus diperhatikan sebelum bertindak.

Kotak abu-abu berhuruf komputer berisi perintah yang diketik di server.
Baris yang diawali tanda # adalah keterangan, bukan perintah.
docker compose ps

1.3 Bagian Mana yang Perlu Dibaca

Peran	Bab yang disarankan
Pimpinan / non-teknis	Bab 1 (Pendahuluan), Bab 8 (Risiko), dan kotak "Singkatnya" di setiap bab
Support / Helpdesk	Bab 1, Bab 4 (Alur Modul), Bab 7 (Troubleshooting), Bab 9 (SOP)
Developer / Programmer	Semua bab, terutama Bab 3, 4, 5, dan 6
DevOps / Infrastruktur	Bab 2, Bab 3, Bab 6 (Deployment & Backup), Bab 7, Bab 8

Peran	Bab yang disarankan
Admin sistem (superadmin)	Bab 9 (SOP), Bab 7 (Troubleshooting)

Istilah teknis yang asing bisa dicari di bagian **Lampiran: Glosarium Istilah** di akhir dokumen.

1.4 Tentang SIMASTER

SINGKATNYA

1. SIMASTER adalah aplikasi web (dibuka lewat browser) untuk mencatat dan mengelola semua aset LM FEB UI: gedung, ruangan, barang inventaris, alat pemadam api (APAR), kendaraan, dan stok gudang.
2. Selain mencatat, sistem ini juga mengurus semua kegiatan yang berhubungan dengan aset: peminjaman, permintaan, pembelian, pemindahan, pengiriman, pengaduan kerusakan, perawatan, pemusnahan, sampai pemakaian anggaran tahunan (RKAT).

SIMASTER mengelola seluruh "siklus hidup" aset fisik institusi, mulai dari aset dicatat, dipakai, dipinjam, dipindah, dirawat, hingga dimusnahkan. Cakupannya:

- **Inventarisasi aset** — pencatatan aset inventaris, APAR, dan kendaraan beserta kondisi dan lokasinya
- **Manajemen infrastruktur** — data gedung, ruangan, gambar, dan denah
- **Transaksi operasional** — peminjaman, pemindahan, pengadaan, permintaan barang gudang, ekspedisi, pengaduan kerusakan, dan maintenance
- **Manajemen gudang** — stok barang, transaksi masuk/keluar, stok opname, dan rekap bulanan otomatis
- **Anggaran (RKAT)** — perencanaan anggaran tahunan per pos kegiatan, pencatatan realisasi, dan laporan serapan
- **Laporan & audit** — laporan tahunan, laporan pemusnahan, audit log seluruh perubahan data

Informasi	Keterangan
Nama aplikasi	SIMASTER — Sistem Manajemen Aset
Pemilik	General Affair, LM FEB UI
Alamat aplikasi	https://simaster.lmfebui.com/
Repository kode	pinpinpinaaaaaa/manajemen-aset
Teknologi inti	Laravel 12, PHP 8.2, MySQL 8.0, Docker
Versi dokumen	September 2026

1.5 Siapa yang Menggunakan Aplikasi Ini

Tipe Pengguna	Cara Akses	Keterangan
Staff / Admin internal	Login dengan akun — /login	Akses dikontrol per role + menu
Superadmin	Login dengan akun — akses penuh	Approve transaksi gudang, manage user/role/menu
Pengguna eksternal	Form publik — tanpa login	Pengajuan peminjaman, ekspedisi, pengaduan, permintaan barang/kendaraan

Kontrol akses berbasis **RBAC**: setiap role memiliki daftar menu yang diizinkan. Akses bisa di-override per user individual.

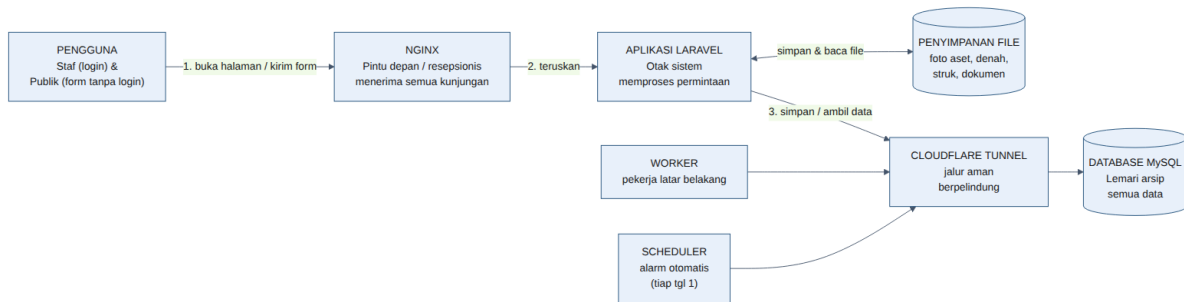
Penjelasan sederhana

Ada dua "pintu masuk" ke sistem. Staf internal masuk lewat halaman login (<https://simaster.lmfebui.com/login>) dan hanya bisa membuka menu sesuai jabatannya. Pihak luar cukup mengisi formulir publik (<https://simaster.lmfebui.com/menu-form>) tanpa perlu akun.

1.6 Cara Kerja Sistem dalam Satu Gambar

Bayangkan sistem ini seperti sebuah kantor pelayanan:

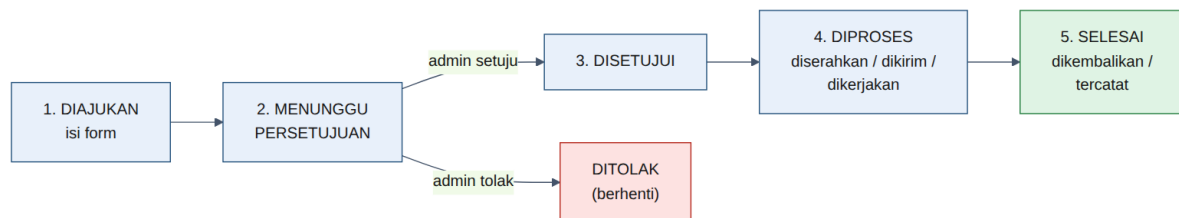
Bagian sistem	Ibaratnya	Tugasnya
Nginx	Resepsionis di pintu depan	Menerima semua pengunjung (browser) dan mengarahkan ke bagian yang tepat
Aplikasi Laravel	Petugas pelayanan / otak	Memeriksa hak akses, memproses pengajuan, menyusun halaman
Database MySQL	Lemari arsip	Menyimpan semua data: aset, transaksi, user, anggaran
Penyimpanan file	Rak dokumen	Menyimpan foto aset, denah gedung, struk, dan lampiran
Worker	Staf back-office	Mengerjakan tugas di belakang layar agar pengguna tidak perlu menunggu
Scheduler	Jam alarm	Menjalankan tugas rutin otomatis, misalnya rekap stok tiap tanggal 1
Cloudflare Tunnel	Lorong rahasia berpintu kartu	Jalur aman dari aplikasi ke database di server kantor



Gambar: Gambaran sederhana cara kerja sistem

1.7 Pola Umum Setiap Pengajuan

Hampir semua fitur transaksi (peminjaman, permintaan, pengadaan, pemindahan, dan lainnya) mengikuti lima langkah yang sama. Jika memahami pola ini, sebagian besar alur di Bab 4 akan mudah diikuti.



Gambar: Siklus pengajuan yang berlaku di hampir semua modul

1.8 Daftar Modul (Fitur) Utama

Modul	Untuk apa?	Perlu login?
Infrastruktur	Data gedung dan ruangan, lengkap dengan foto dan denah	Ya
Aset Inventaris	Daftar semua barang milik institusi, kondisi, lokasi, dan riwayatnya	Ya
APAR	Data alat pemadam api ringan dan riwayat pemeriksaannya	Ya
Kendaraan	Data kendaraan dinas dan permintaan pemakaian kendaraan	Formulir permintaan: tidak
Gudang	Stok barang habis pakai, barang masuk/keluar, stok opname, rekap bulanan	Ya
Peminjaman	Peminjaman aset dan ruangan	Formulir pengajuan: tidak
Permintaan & Pengadaan	Permintaan barang dari gudang dan pembelian barang/jasa ke vendor	Formulir pengajuan: tidak
Pemindahan	Memindahkan aset antar gedung/ruangan	Ya
Ekspedisi	Pengiriman barang/dokumen keluar, cetak label dan tanda terima	Formulir pengajuan: tidak
Pengaduan Kerusakan	Laporan kerusakan fasilitas dari siapa saja	Formulir laporan: tidak

Modul	Untuk apa?	Perlu login?
Maintenance	Pencatatan perawatan dan perbaikan aset beserta biayanya	Ya
Laporan	Laporan pemusnahan aset dan laporan tahunan	Ya
Anggaran RKAT	Rencana anggaran tahunan dan pemakaiannya (terisi otomatis)	Ya
Admin	Kelola user, jabatan (role), menu, dan audit log	Ya (superadmin)

1.9 Hal Penting yang Perlu Diketahui Sejak Awal

Perhatian: 4 hal penting

1. Belum ada backup otomatis. Jika server rusak, data bisa hilang permanen. Lihat subbab 6.6 dan Bab 8 (Risiko R2).
2. Formulir publik belum memakai CAPTCHA, sehingga rawan diisi spam oleh bot. Lihat Bab 8 (Risiko R1).
3. Pengujian otomatis (automated test) masih sangat sedikit. Setiap perubahan kode perlu dicek manual dengan teliti sebelum dipasang ke server asli.
4. Beberapa standar (backup otomatis, CAPTCHA, email notifikasi, arsip audit log) sudah ditetapkan tetapi belum terpasang di sistem. Daftar lengkap dan statusnya ada di Bab 8, Bagian A.

TEKNOLOGI YANG DIPAKAI

SINGKATNYA

1. Bab ini menjawab pertanyaan "aplikasi ini dibuat pakai apa?". Bayangkan sebuah rumah: PHP dan Laravel adalah bahan bangunan dan kerangkanya, MySQL adalah lemari arsip tempat semua data disimpan, dan Docker adalah "kontainer" yang membungkus seluruh aplikasi supaya bisa dipindahkan ke server mana pun dan tetap berjalan sama persis.
2. Bab ini paling berguna saat tim perlu upgrade versi, membeli atau menyewa server, atau mencari programmer yang menguasai teknologinya.

2.1 Bahasa & Framework

Komponen	Versi	Kegunaan
PHP	^8.2	Runtime backend
Laravel	^12.0	Framework MVC utama
Node.js	22 (Alpine, build stage)	Build frontend assets (Vite)

2.2 Library PHP (Production)

Penjelasan sederhana

Library adalah "komponen siap pakai" buatan pihak lain, supaya tim tidak perlu membuat semuanya dari nol. Contoh: DomPDF dipakai untuk membuat file PDF, Maatwebsite Excel untuk membuat file Excel.

Library	Versi	Kegunaan
laravel/framework	^12.0	Core framework, routing, Eloquent ORM, queue, scheduler
barryvdh/laravel-dompdf	^3.1	Generate PDF (laporan, label ekspedisi, tanda terima)
maatwebsite/excel	^3.1	Export/import Excel (.xlsx) — Maatwebsite Excel berbasis PhpSpreadsheet
intervention/image	^3.11	Resize & proses gambar upload (foto aset, denah gedung)
laravel/ui	^4.6	Auth scaffolding (login, register, middleware auth)
laravel/tinker	^2.10.1	REPL untuk debugging di environment lokal/staging

2.3 Library PHP (Development)

Library	Versi	Kegunaan
phpunit/phpunit	^11.5.3	Unit & feature testing
laravel/pint	^1.24	Code formatter (PHP-CS-Fixer wrapper)

Library	Versi	Kegunaan
laravel/pail	^1.2.2	Log tailing real-time di terminal
laravel/sail	^1.41	Dev environment berbasis Docker (opsional, tidak dipakai di production)
fakerphp/faker	^1.23	Generate data dummy untuk factories/seeder
nunomaduro/collision	^8.6	Error reporting lebih informatif di CLI
mockery/mockery	^1.6	Mocking objects di unit test

2.4 Library Frontend

Penjelasan sederhana

Frontend adalah bagian yang dilihat pengguna di browser: tampilan, tombol, warna, dan tata letak. Bootstrap dan Tailwind adalah "perangkat desain" untuk membuat tampilan rapi.

Library	Versi	Kegunaan
Vite	^7.0.7	Module bundler & dev server — bundle CSS/JS ke public/build/
laravel-vite-plugin	^2.0.0	Integrasi Vite dengan Laravel (hot reload, manifest)
Bootstrap	^5.2.3	Komponen UI (grid, modal, badge, btn, alert, table)
@popperjs/core	^2.11.6	Dependency Bootstrap (tooltip, dropdown)
Tailwind CSS	^4.0.0	Utility classes tambahan di samping Bootstrap
@tailwindcss/vite	^4.0.0	Plugin Vite untuk Tailwind v4
Sass	^1.56.1	Pre-processor CSS (resources/sass/)
Axios	^1.11.0	HTTP client JavaScript (AJAX cek ketersediaan, dll)
Prettier	^3.8.1 + prettier-plugin-blade	Code formatting Blade templates

2.5 Database & Storage

Penjelasan sederhana

Data teks (nama aset, transaksi, user) disimpan di database MySQL. File (foto, PDF, struk) disimpan di folder penyimpanan server, bukan di database dan bukan di cloud.

Komponen	Versi	Kegunaan
MySQL	8.0 (Docker image)	Database utama production
SQLite	(bawaan PHP)	Alternatif untuk development lokal (database/database.sqlite)
File storage lokal	—	Upload foto/dokumen di storage/app/public/ — tidak menggunakan cloud storage

Driver Laravel yang dipakai:

```
SESSION_DRIVER=database → tabel sessions
QUEUE_CONNECTION=database → tabel jobs, failed_jobs
CACHE_STORE=database → tabel cache
```

2.6 Infrastruktur & Deploy

Penjelasan sederhana

Sistem berjalan di beberapa "kotak" (container) terpisah yang masing-masing punya satu tugas: pintu depan (nginx), otak aplikasi (app), pekerja latar belakang (worker), alarm terjadwal (scheduler), dan database. Jika satu kotak mati, gejalanya berbeda-beda. Lihat Bab 7 (Troubleshooting).

Komponen	Image / Teknologi	Fungsi
PHP-FPM	php:8.2-fpm	Proses PHP — container app
Nginx	nginx:1.26-alpine	Web server + static file serving — container nginx
MySQL	mysql:8.0	Database — container mysql (dev) / eksternal (prod)
Queue Worker	image app	Proses queue database — container worker
Scheduler	image app	Jalankan cron rekap:bulanan — container scheduler
Docker	Engine ≥ 24 + Compose plugin	Orkestrasi semua container
Cloudflare Zero Trust	cloudflared (binary di image)	Tunnel terenkripsi ke database on-premise (production)

Docker build stages (Dockerfile):

1. **assets** (Node 22 Alpine) — `npm run build` → **public/build/**
2. **runtime** (PHP 8.2-FPM) — install Composer deps + salin kode + hasil Vite
3. **nginx-stage** (Nginx 1.26 Alpine) — serve **public/** + proxy ke PHP-FPM

PHP extensions yang diinstall di image: `pdo_mysql`, `gd`, `zip`, `bcmath`, `intl`, `mbstring`, `xml`, `dom`, `exif`, `opcache`

2.7 Kebutuhan Server

Penjelasan sederhana

Kolom MINIMUM artinya aplikasi bisa berjalan, tetapi bisa terasa lambat saat banyak orang memakai atau saat export data besar. Kolom REKOMENDASI adalah spesifikasi agar aplikasi berjalan lancar dan masih punya ruang untuk bertambahnya data beberapa tahun ke depan. Untuk server baru, gunakan kolom REKOMENDASI.

Spesifikasi berikut adalah **standar yang ditetapkan** untuk SIMASTER. Angkanya dihitung dari susunan aplikasi (5 container: nginx, app/php-fpm, worker, scheduler, cloudflared) dan bagian yang paling berat: export Excel/PDF (DomPDF & PhpSpreadsheet memakai memori besar) serta upload foto. Asumsi beban: sampai **±50 pengguna aktif bersamaan** dan **±5.000–20.000 data aset**.

Di production, aplikasi dan database berada di **dua server terpisah**: server aplikasi (Docker) dan server database MySQL on-premise yang tersambung lewat Cloudflare Tunnel.

2.7.1 Server Aplikasi (Docker)

Komponen	Minimum (bisa jalan)	Rekomendasi (lancar)	Alasan
CPU	2 vCPU	4 vCPU	5 container jalan bersamaan; export PDF/Excel memakan CPU tinggi selama beberapa detik
RAM	4 GB	8 GB	Tiap proses PHP bisa memakai sampai 256 MB saat export; ditambah worker, scheduler, nginx, dan OS
Penyimpanan	40 GB SSD	80 GB SSD	Image Docker ±2 GB, log, cache, dan file upload (foto aset, denah, struk) yang terus bertambah
Jenis disk	SSD	SSD / NVMe	HDD membuat halaman dan export terasa lambat
OS	Ubuntu Server 22.04 / 24.04 LTS (64-bit)	Ubuntu Server 24.04 LTS (64-bit)	Versi LTS mendapat update keamanan jangka panjang
Software	Docker Engine ≥ 24 + Compose v2	Docker Engine terbaru + Compose v2	Dibutuhkan untuk menjalankan semua container
Swap	2 GB	4 GB	Cadangan saat RAM penuh agar aplikasi tidak mati mendadak

2.7.2 Server Database (MySQL 8.0 On-Premise)

Komponen	Minimum (bisa jalan)	Rekomendasi (lancar)	Alasan
CPU	2 vCPU	4 vCPU	Query laporan, rekap bulanan, dan audit log
RAM	4 GB	8 GB	Sebagian besar RAM dipakai cache MySQL (<code>innodb_buffer_pool_size</code>) agar data sering dipakai tidak dibaca ulang dari disk
Penyimpanan data	50 GB SSD	100 GB SSD	Tabel <code>audit_logs</code> tumbuh terus karena semua perubahan data dicatat

Komponen	Minimum (bisa jalan)	Rekomendasi (lancar)	Alasan
Disk backup	100 GB (disk terpisah)	200 GB di NAS / disk eksternal	Menyimpan backup harian 30 hari; wajib terpisah dari disk data
OS	Ubuntu Server 22.04 / 24.04 LTS	Ubuntu Server 24.04 LTS	—

2.7.3 Jika Aplikasi & Database di Satu Server

Untuk instalasi sederhana (misalnya server cadangan atau staging), semua bisa dijalankan di satu server dengan `docker-compose.yml`:

Komponen	Minimum	Rekomendasi
CPU	4 vCPU	4–8 vCPU
RAM	8 GB	16 GB
Penyimpanan	80 GB SSD	150 GB SSD + disk backup terpisah

2.7.4 Jaringan

Komponen	Standar
Koneksi internet server aplikasi	Minimal 20 Mbps, disarankan 50 Mbps ke atas (upload/download simetris lebih baik)
Koneksi internet server database (on-premise)	Minimal 20 Mbps dan stabil; tunnel Cloudflare putus = aplikasi tidak bisa membaca data
Latensi aplikasi ↔ database	Idealnya di bawah 30 ms; di atas 100 ms halaman mulai terasa lambat
Port terbuka ke publik	Hanya 80 dan 443 (HTTPS). Port database 3306 tidak dibuka ke internet
Domain & SSL	simaster.lmfebui.com dengan sertifikat HTTPS aktif

2.7.5 Pengaturan Agar Aplikasi Lancar

Pengaturan	Nilai standar	Keterangan
PHP <code>memory_limit</code>	256M (512M jika export data besar sering gagal)	Batas memori per proses PHP
PHP <code>max_execution_time</code>	120 detik	Memberi waktu cukup untuk export PDF/Excel besar
PHP-FPM <code>pm.max_children</code>	10 (RAM 4 GB) / 20 (RAM 8 GB)	Jumlah proses PHP paralel; terlalu besar bisa membuat RAM habis
OPcache	Aktif (sudah terpasang di image)	Mempercepat eksekusi PHP
Nginx <code>client_max_body_size</code>	64M (sudah diatur)	Batas ukuran upload
MySQL <code>innodb_buffer_pool_size</code>	±50–60% RAM server database (2 GB untuk RAM 4 GB, 4–5 GB untuk RAM 8 GB)	Cache utama MySQL, paling berpengaruh ke kecepatan

Pengaturan	Nilai standar	Keterangan
MySQL <code>max_connections</code>	150	Cukup untuk app, worker, scheduler, dan admin
Backup	Harian pukul 02:00, simpan 30 hari	Lihat Bab 6 (Deployment & Backup)

2.7.6 Tanda Server Perlu Di-upgrade

Gejala	Tindakan
Pemakaian RAM di atas 80% hampir sepanjang hari	Tambah RAM
CPU di atas 70% dalam waktu lama, halaman terasa lambat	Tambah vCPU
Sisa disk di bawah 20%	Tambah disk atau arsipkan audit log / file lama
Export PDF/Excel sering gagal atau error memori	Naikkan <code>memory_limit</code> ke 512M, lalu tambah RAM bila perlu

2.8 Tools Build, Test & Format

Perintah	Tool	Fungsi
<code>npm run build</code>	Vite	Bundle CSS/JS untuk production
<code>npm run dev</code>	Vite dev server	Hot reload saat development
<code>composer run test</code>	PHPUnit ^11	Jalankan semua test
<code>php artisan test --filter NamaTest</code>	PHPUnit	Jalankan satu test spesifik
<code>./vendor/bin/pint</code>	Laravel Pint	Format/lint kode PHP
<code>composer run dev</code>	concurrently	Jalankan server + queue + log + vite bersamaan (dev)

2.9 Layanan Pihak Ketiga

Layanan	Kegunaan	Konfigurasi
Cloudflare Zero Trust	Tunnel terenkripsi dari container ke database MySQL on-premise	<code>CF_ACCESS_HOSTNAME</code> , <code>CF_ACCESS_CLIENT_ID</code> , <code>CF_ACCESS_CLIENT_SECRET</code> di <code>.env</code>
GitHub Container Registry (ghcr.io)	Opsional: hosting Docker image untuk deployment produksi	<code>GITHUB_USER</code> di <code>.env</code> ; hanya jika pakai <code>docker-compose.prod.yml</code> mode [A]
Email (SMTP)	Notifikasi email pengajuan disetujui/ditolak. Standar: diaktifkan di production memakai akun email resmi LM FEB UI (lihat Bab 8, A3)	<code>MAIL_MAILER=smtp</code> dan <code>MAIL_*</code> di <code>.env</code>

ARSITEKTUR: BAGAIMANA SISTEM DISUSUN

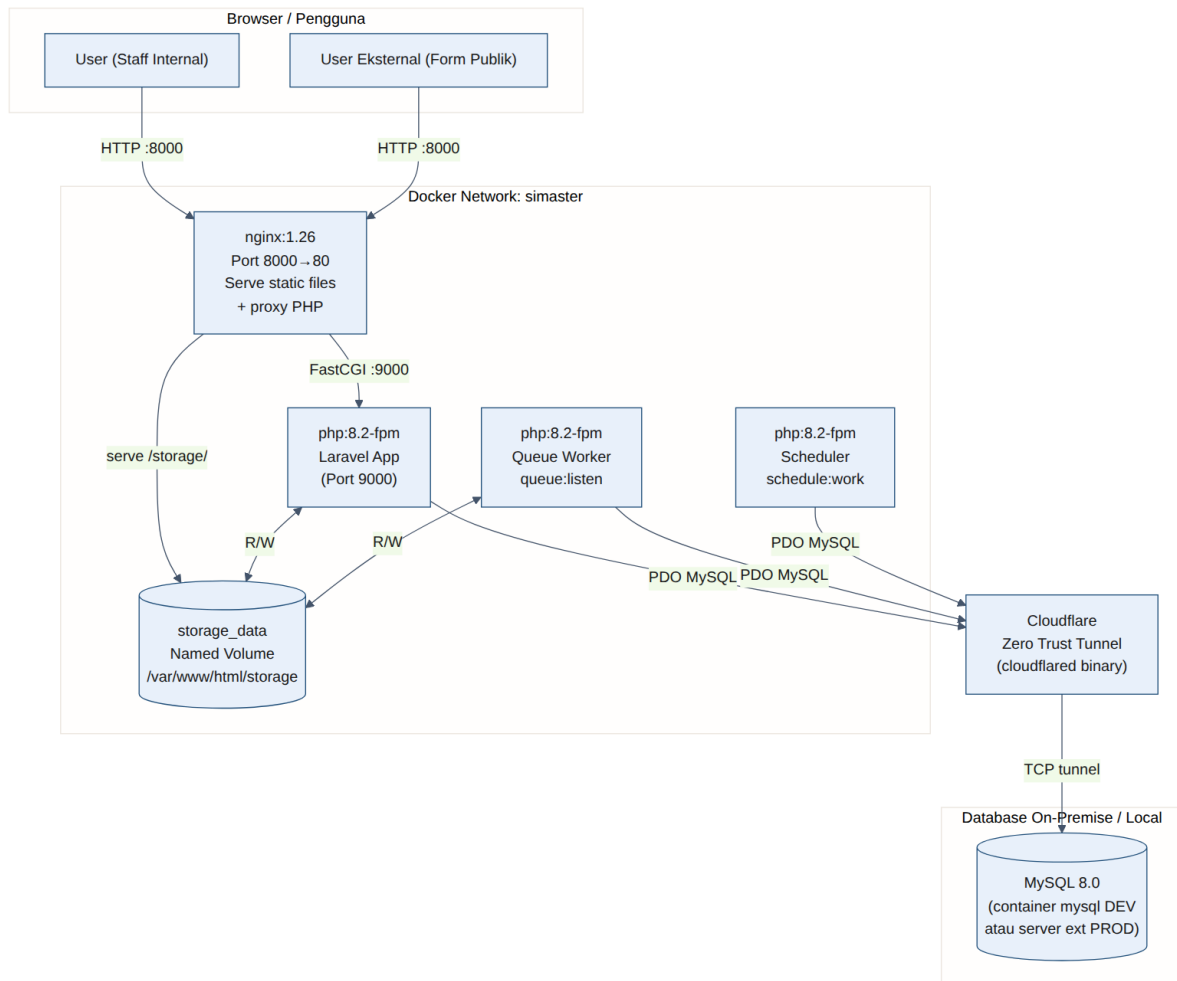
SINGKATNYA

1. Arsitektur adalah "denah bangunan" sistem: bagian apa saja yang ada dan bagaimana mereka saling terhubung. Gampangnya seperti restoran: Nginx adalah pelayan di pintu depan yang menerima tamu, aplikasi Laravel adalah dapur yang memasak pesanan, database adalah gudang bahan, worker adalah staf yang mengerjakan pekerjaan di belakang layar, dan scheduler adalah jam alarm yang menyalakan tugas rutin.
2. Cloudflare Tunnel adalah "jalur pipa tertutup" yang menghubungkan aplikasi ke database di kantor tanpa membuka pintu database ke internet.

3.1 Diagram Arsitektur (Production)

Penjelasan sederhana

Cara membaca diagram: pengguna membuka aplikasi lewat browser, lalu permintaannya diterima Nginx, diteruskan ke aplikasi Laravel, dan aplikasi mengambil atau menyimpan data di database melalui jalur aman Cloudflare. File upload (foto, dokumen) disimpan di "storage_data".



Gambar: Diagram Arsitektur (Production)

Catatan

Dev vs Prod:

- **Dev** (`docker-compose.yml`): MySQL berjalan sebagai container `mysql` di network yang sama. `CF_ACCESS_CLIENT_ID` tidak diset → cloudflared tidak dijalankan, `DB_HOST=mysql`.
- **Prod** (`docker-compose.prod.yml`): Tidak ada container MySQL. Database di server on-premise. `DB_HOST=127.0.0.1` karena cloudflared membuat TCP proxy lokal.

3.2 Komponen & Fungsinya

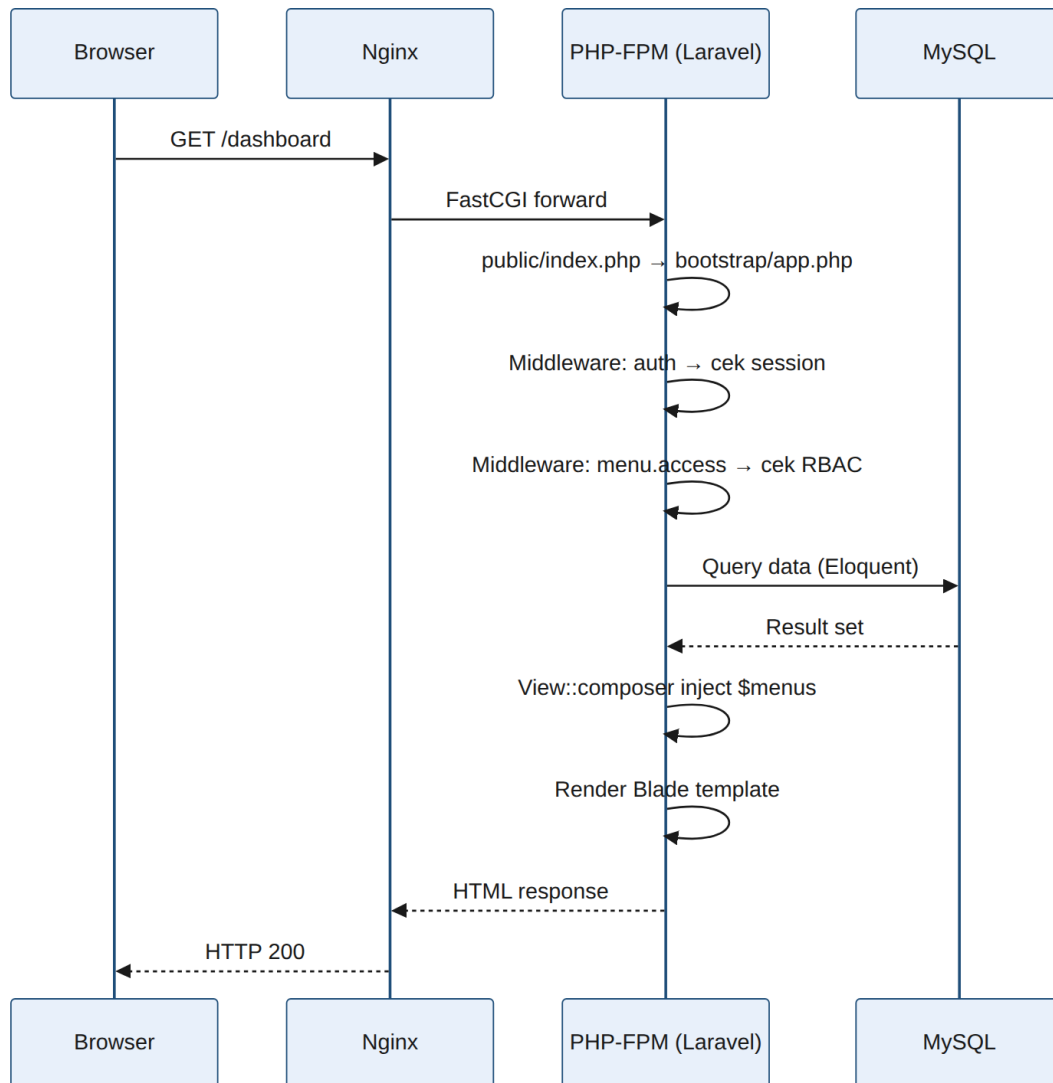
Komponen	File / Image	Tanggung Jawab
Nginx	<code>docker/nginx.conf</code> , image <code>nginx-stage</code>	Menerima request HTTP, serve <code>public/build/</code> (assets) dan <code>storage/</code> (file upload), forward PHP ke php-fpm
PHP-FPM App	<code>Dockerfile</code> target <code>runtime</code>	Menjalankan Laravel — routing, controller, Eloquent, render view

Komponen	File / Image	Tanggung Jawab
Queue Worker	Container <code>worker</code> , command <code>queue:listen</code>	Memproses job asinkron dari tabel <code>jobs</code> (database driver)
Scheduler	Container <code>scheduler</code> , command <code>schedule:work</code>	Menjalankan cron Laravel — saat ini hanya <code>rekap:bulanan</code> tiap tgl 1 jam 00:00
MySQL	<code>mysql:8.0</code> / server eksternal	Penyimpanan data utama — semua tabel transaksional, session, queue, cache
Named Volume <code>storage_data</code>	Docker volume	Persists file upload (foto aset, struk, dokumen) agar tidak hilang saat image di-rebuild
Cloudflare Zero Trust	<code>cloudflared</code> binary di image	Membuat tunnel terenkripsi dari container ke MySQL on-premise (tanpa expose port DB ke internet)

3.3 Alur Request HTTP

Penjelasan sederhana

Urutan kejadian saat seseorang membuka satu halaman: (1) browser meminta halaman, (2) Nginx meneruskan ke aplikasi, (3) aplikasi memeriksa apakah orang ini sudah login, (4) aplikasi memeriksa apakah jabatannya boleh membuka halaman ini, (5) aplikasi mengambil data dari database, (6) aplikasi menyusun halaman dan mengirimnya kembali ke browser. Semua ini biasanya terjadi dalam hitungan milidetik.



Gambar: Alur Request HTTP

3.4 Struktur Folder

Penjelasan sederhana

Ini adalah susunan folder kode program. Tim non-programmer tidak perlu menghafalnya. Yang penting diingat: folder storage/logs berisi catatan error, dan storage/app/public berisi file upload pengguna.

```

manajemen-aset/
├── app/
│   ├── Console/Commands/ ← Artisan commands (RekapBulanan)
│   ├── Exports/ ← Kelas export Excel (Maatwebsite) & PDF helper per-modul
│   ├── Http/
│   │   ├── Controllers/ ← ~30 controller, 1 file per modul
│   │   └── Middleware/ ← CheckMenuAccess (RBAC)
│   ├── Models/ ← ~57 Eloquent model, semua extend BaseModel
│   ├── Observers/ ← UserObserver
│   └── Providers/ ← AppServiceProvider (View::composer untuk $menus)
    
```

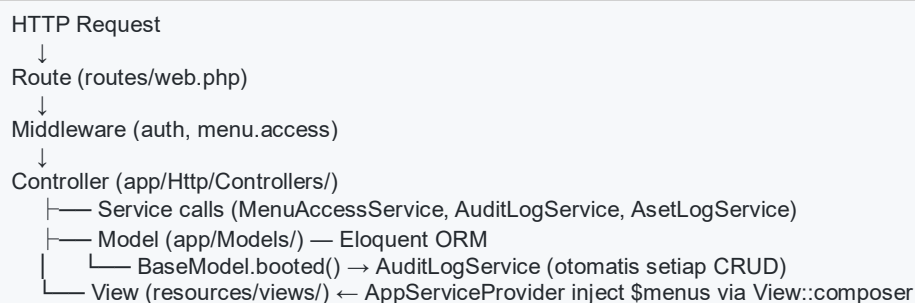
Services/	← MenuAccessService, AuditLogService, AsetLogService
helpers.php	← Global helper functions
bootstrap/	
app.php	← Entry config Laravel: routing, middleware alias, schedule
config/	
coa.php	← Chart of Accounts (kategori anggaran bertingkat)
*.php	← Config standar Laravel (database, queue, cache, dll)
database/	
migrations/	← ~60 migration file (additive-only di production)
seeders/	← Seeder untuk data awal (role, menu, superadmin)
database.sqlite	← Database dev lokal (SQLite)
docker/	
entrypoint.sh	← Init: cloudflared → storage → migrate → cache → php-fpm
nginx.conf	← Konfigurasi Nginx (upload 64MB, cache static, proxy PHP)
docs/	← Dokumentasi teknis ini
public/	
index.php	← Entry point web (satu-satunya PHP yang di-hit langsung)
build/	← Output Vite (CSS/JS ter-bundle + hash nama file)
resources/	
css/	← styles.css (custom) + app.css (Tailwind)
js/	← app.js + modul JS kustom
sass/	← SCSS files
views/	← Blade templates, 1 subfolder per modul
routes/	
web.php	← Semua route (publik + auth-protected)
storage/	
app/public/	← File upload (symlink dari public/storage)
framework/	← Cache view, session, compiled routes
logs/	← Laravel log files
tests/	
Feature/	← Feature tests (ApprovalWorkflowTest)
Unit/	← Unit tests (placeholder)
Dockerfile	← Multi-stage build: assets → runtime → nginx-stage
docker-compose.yml	← Dev: 5 services + MySQL container
docker-compose.prod.yml	← Prod: 4 services, tanpa MySQL (eksternal via CF tunnel)

3.5 Pola Desain

Penjelasan sederhana

Pola desain adalah "cara menata kode" supaya rapi dan mudah dirawat. Sistem ini memakai pola MVC: Model (mengurus data), View (mengurus tampilan), Controller (mengatur alur di antara keduanya).

3.5.1 MVC + Service Layer

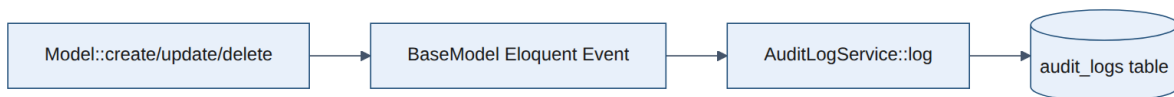


3.5.2 BaseModel & Audit Otomatis

Penjelasan sederhana

Seperti CCTV untuk data: setiap kali ada data yang ditambah, diubah, atau dihapus, sistem otomatis mencatat siapa yang melakukannya, kapan, dari komputer mana, dan isi data sebelum serta sesudah perubahan. Catatan ini bisa dilihat di menu Audit Log.

Semua model (kecuali `AuditLog` sendiri) extend `BaseModel` (`app/Models/BaseModel.php`). `BaseModel::booted()` mendaftarkan Eloquent event listener untuk `created`, `updated`, `deleted` yang otomatis memanggil `AuditLogService::log()`.

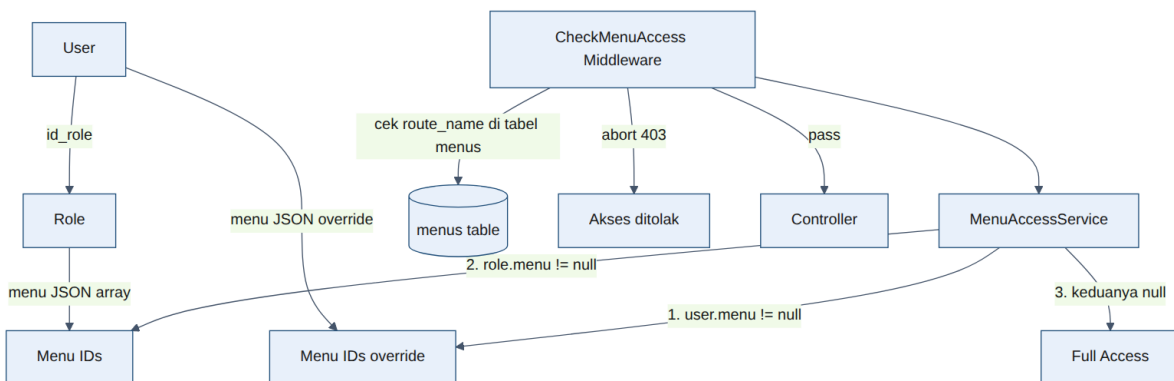


Gambar: BaseModel & Audit Otomatis

3.5.3 RBAC (Role-Based Access Control)

Penjelasan sederhana

RBAC mirip kartu akses gedung. Setiap jabatan (role) punya daftar "ruangan" (menu) yang boleh dimasuki. Jika perlu, satu orang bisa diberi kartu khusus (override) yang berbeda dari jabatannya. Jika seseorang membuka menu yang tidak ada di kartunya, sistem menampilkan error 403 (akses ditolak).



Gambar: RBAC (Role-Based Access Control)

Hirarki akses (`app/Services/MenuAccessService.php`):

1. Jika `user.menu != null` → pakai override individual
2. Jika `role.menu != null` → pakai menu dari role
3. Jika keduanya `null` → **full access** (khusus superadmin tanpa pembatasan)

Middleware `menu.access` di-alias di `bootstrap/app.php` → `App\Http\Middleware\CheckMenuAccess`.

3.6 Modul-Modul Utama

Penjelasan sederhana

Tabel ini memetakan setiap fitur ke file programnya. Berguna bagi programmer saat mencari lokasi kode yang harus diperbaiki.

Modul	Controller	Model Utama
Infrastruktur	GedungController, RuanganController	Gedung, Ruangan, GedungGambar, GedungDenah, RuanganGambar
Aset Inventaris	AsetController	Aset, AsetLog, JenisBarang
APAR	AparController	Apar, AparLog
Kendaraan	KendaraanController	Kendaraan
Gudang	GudangController	GudangBarang, GudangTransaksi, GudangTransaksiDetail, GudangTransaksiCharge, GudangRekapBulanan, GudangOpnameHeader, GudangOpnameDetail
Peminjaman	PeminjamanAsetController, PeminjamanRuanganController	PeminjamanAset, PeminjamanAsetDetail, PeminjamanRuangan, PeminjamanRuanganDetail, PeminjamanRuanganAset, PeminjamanRuanganKonsumsi
Permintaan & Pengadaan	PermintaanBarangGudangController, PengadaanBarangJasaController	PermintaanBarangGudang, PengadaanBarangJasa, PengadaanBarangJasaDetail, PengadaanBarangJasaFile
Kendaraan (Permintaan)	PermintaanKendaraanController	PermintaanKendaraan, PermintaanKendaraanDetail, PermintaanKendaraanItem
Pemindahan	PemindahanAsetController	PemindahanAset, PemindahanAsetDetail
Ekspedisi	EkspedisiController	Ekspedisi, EkspedisiBarang, EkspedisiDokumen, EkspedisiPengiriman
Pengaduan Kerusakan	PengaduanKerusakanController	PengaduanKerusakan, PengaduanKerusakanDetail
Maintenance	MaintenanceController	Maintenance, MaintenanceDetail
Pemusnahan	LaporanPemusnahanController	LaporanPemusnahan
Laporan Tahunan	LaporanTahunanController	LaporanTahunan, LaporanTahunanDetail, LaporanTahunanSummary
Anggaran RKAT	AnggaranController	RkatAnggaran, RkatRealisasi
Admin	UserController, RolesController, MenuController	User, Role, Menu
Audit	AuditLogController	AuditLog
Vendor	VendorController	Vendor

ALUR KERJA SETIAP MODUL

SINGKATNYA

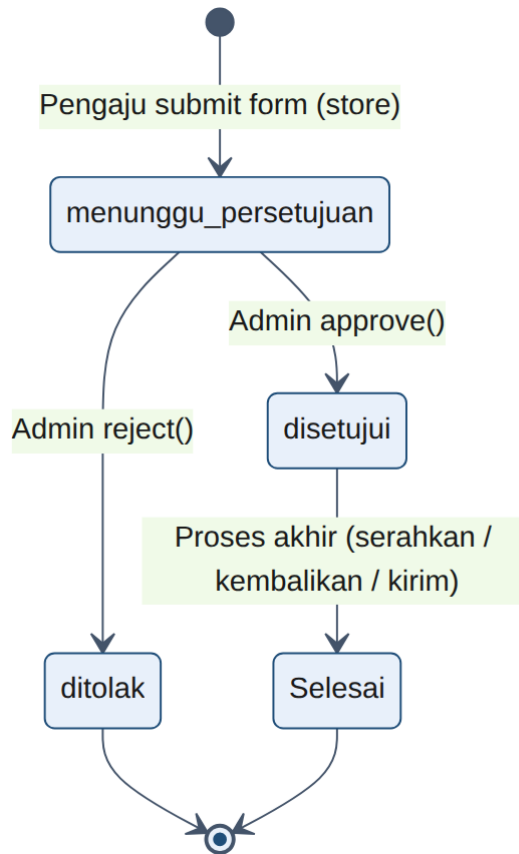
1. Hampir semua fitur di sistem ini mengikuti pola yang sama: DIAJUKAN, lalu MENUNGGU PERSETUJUAN, lalu DISETUJUI atau DITOLAK, lalu DIPROSES, dan terakhir SELESAI. Setiap modul di bawah ini punya ringkasan "Dalam bahasa sederhana" sebelum diagram teknisnya.
2. Cara membaca diagram urutan (sequence diagram): kotak di bagian atas adalah pihak yang terlibat (orang atau bagian sistem). Panah dibaca dari atas ke bawah sesuai urutan kejadian. Panah garis penuh berarti "mengirim/meminta", panah putus-putus berarti "membalas". Kotak kuning adalah catatan.

Semua modul transaksional mengikuti pola yang sama. Bagian ini mendokumentasikan alur tiap modul beserta diagram sekuens.

4.1 Pola Umum Modul Transaksional

Penjelasan sederhana

Setiap pengajuan punya dua "label status". Label pertama (decision_status) menjawab "sudah disetujui atau belum?". Label kedua (status) menjawab "sudah sampai tahap mana pengerjaannya?".



Gambar: Pola Umum Modul Transaksional

Dua field status selalu ada di tabel transaksional:

- **decision_status:** menunggu_persetujuan → disetujui / ditolak
- **status:** Belum Diproses → Sedang Diproses → Selesai

4.2 Peminjaman Aset

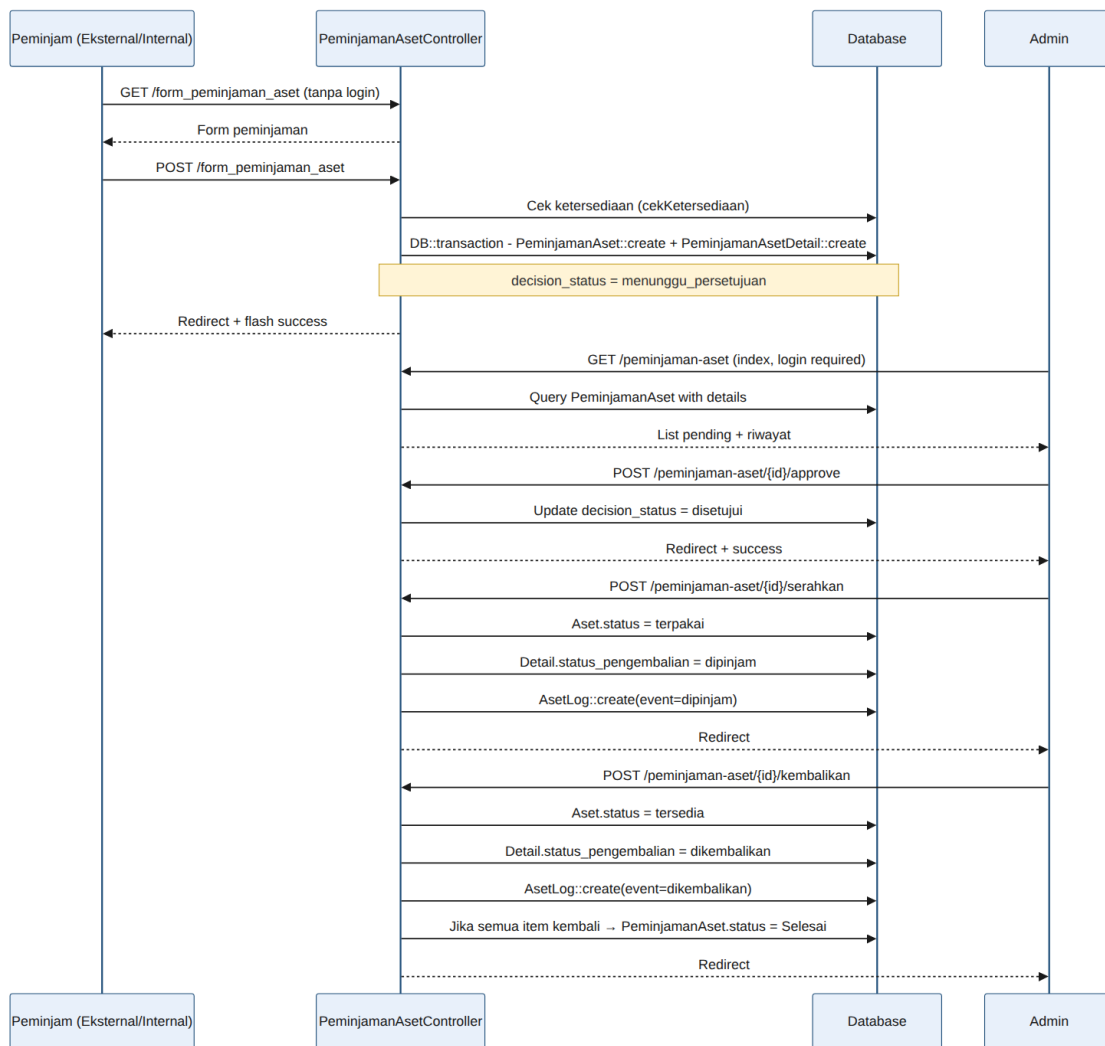
Dalam bahasa sederhana

1. Peminjam mengisi form online (tidak perlu login). Sistem otomatis memeriksa apakah barang tersedia pada tanggal yang diminta.
2. Pengajuan tersimpan dengan status "menunggu persetujuan".
3. Admin melihat daftar pengajuan, lalu menyetujui atau menolak.
4. Saat barang diserahkan, status aset berubah menjadi "terpakai".
5. Saat barang dikembalikan, aset kembali "tersedia". Jika semua barang sudah kembali, peminjaman otomatis "Selesai".

Controller: PeminjamanAsetController

Model: PeminjamanAset, PeminjamanAsetDetail

Route prefix: /peminjaman-aset



Gambar: Peminjaman Aset

Endpoint AJAX: `GET /cek-ketersediaan` — hitung unit tersedia di rentang tanggal tertentu (pakai `lockForUpdate()` saat `store`).

4.3 Peminjaman Ruangan

Dalam bahasa sederhana

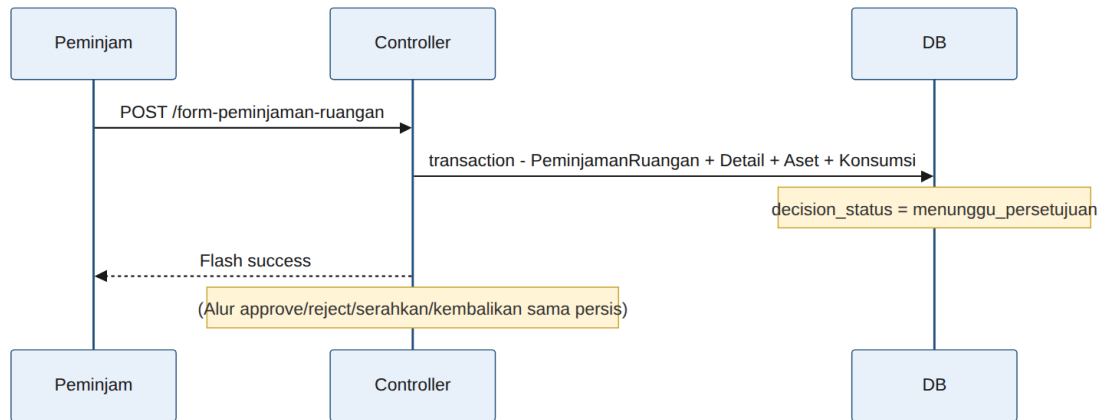
1. Alurnya sama dengan peminjaman aset.
2. Bedanya, peminjam bisa sekaligus meminta perlengkapan di dalam ruangan (misalnya proyektor) dan konsumsi (snack, minuman).

Controller: `PeminjamanRuanganController`

Model: `PeminjamanRuangan`, `PeminjamanRuanganDetail`, `PeminjamanRuanganAset`, `PeminjamanRuanganKonsumsi`

Alur sama dengan Peminjaman Aset, dengan tambahan:

- Detail bisa menyertakan daftar **aset yang ada di ruangan** (**PeminjamanRuanganAset**)
- Bisa menyertakan **konsumsi** (snack, minuman) via **PeminjamanRuanganKonsumsi**
- Endpoint AJAX: **GET /cek-ketersediaan-ruangan**



Gambar: Peminjaman Ruangan

4.4 Permintaan Barang Gudang

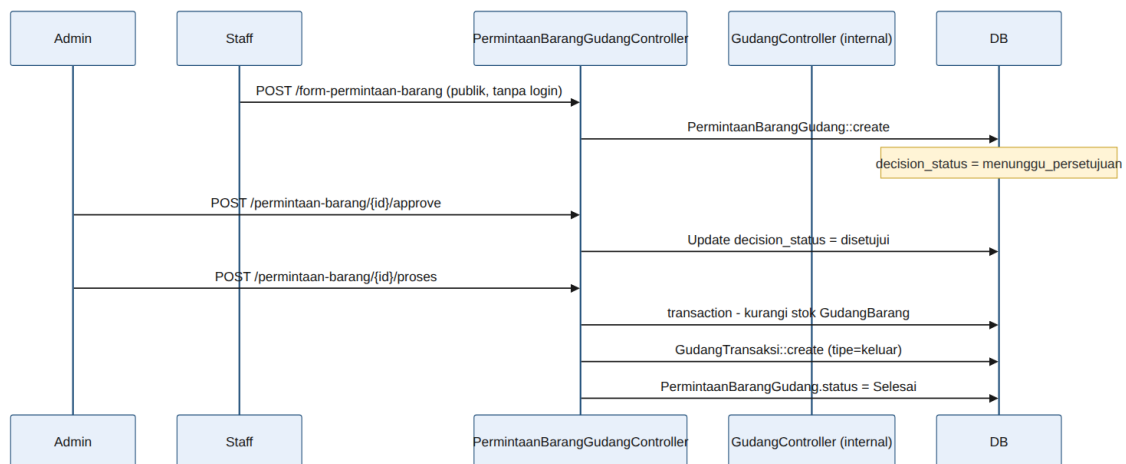
Dalam bahasa sederhana

1. Staf mengajukan permintaan barang dari gudang (misalnya ATK).
2. Admin menyetujui permintaan.
3. Saat diproses, stok gudang otomatis berkurang dan tercatat sebagai "barang keluar".

Controller: **PermintaanBarangGudangController**

Model: **PermintaanBarangGudang**

Alur khas: Setelah disetujui, barang diambil dari stok gudang.



Gambar: Permintaan Barang Gudang

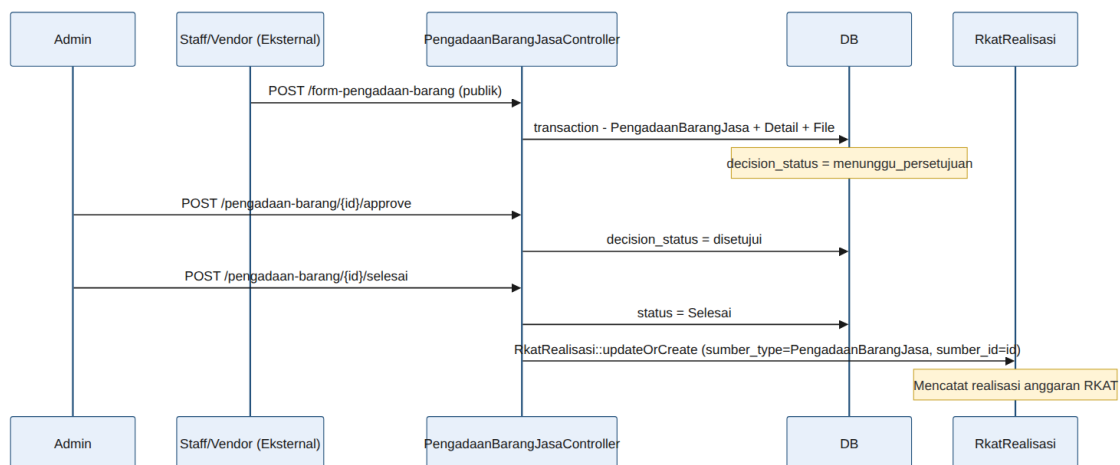
4.5 Pengadaan Barang/Jasa (Pembelian ke Vendor)

Dalam bahasa sederhana

1. Pengaju mengisi form pembelian beserta lampiran (struk, invoice).
2. Admin menyetujui pengajuan.
3. Saat ditandai selesai, biaya pembelian otomatis tercatat sebagai pemakaian anggaran (RKAT). Tidak perlu input ulang.

Controller: `PengadaanBarangJasaController`

Model: `PengadaanBarangJasa`, `PengadaanBarangJasaDetail`, `PengadaanBarangJasaFile`



Gambar: Pengadaan Barang/Jasa (Pembelian ke Vendor)

File upload (struk, invoice) disimpan ke `storage/app/public/pengadaan/`.

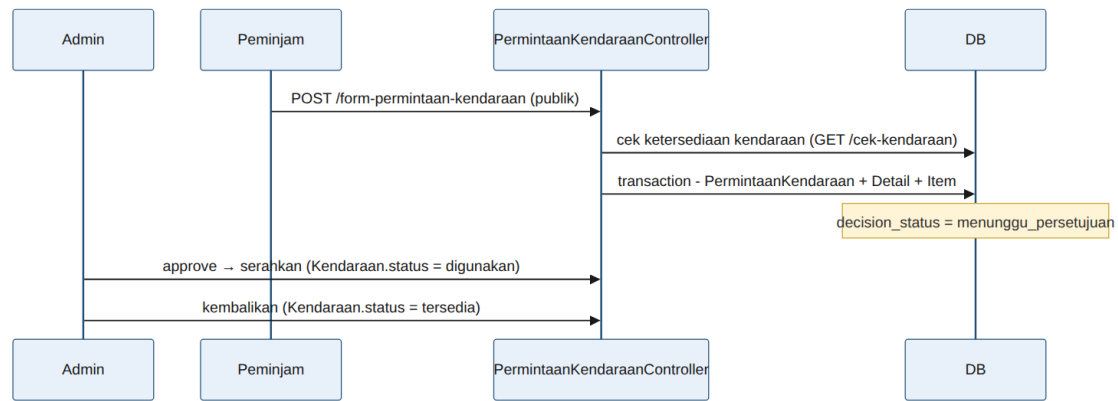
4.6 Permintaan Kendaraan

Dalam bahasa sederhana

1. Pemohon mengisi form. Sistem mengecek kendaraan mana yang tersedia.
2. Admin menyetujui, lalu menyerahkan kendaraan (status kendaraan: "digunakan").
3. Saat dikembalikan, status kendaraan kembali "tersedia".

Controller: `PermintaanKendaraanController`

Model: `PermintaanKendaraan`, `PermintaanKendaraanDetail`, `PermintaanKendaraanItem`



Gambar: Permintaan Kendaraan

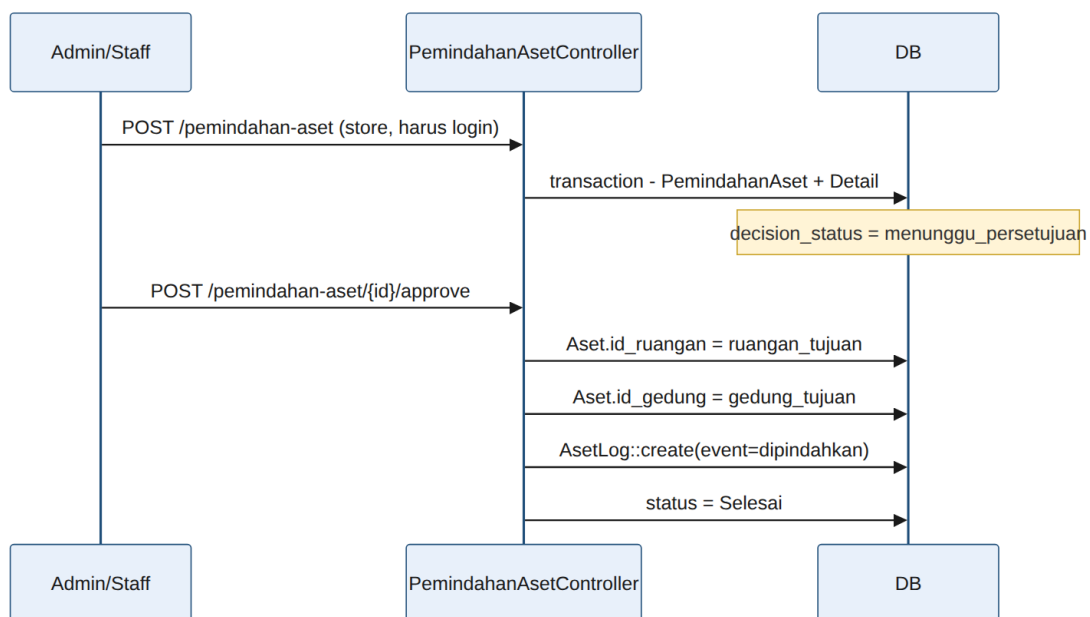
4.7 Pemindahan Aset

Dalam bahasa sederhana

1. Staf (harus login) mengajukan pemindahan aset ke gedung/ruangan lain.
2. Saat disetujui, lokasi aset di sistem otomatis berubah dan riwayat pemindahannya dicatat.

Controller: `PemindahanAsetController`

Model: `PemindahanAset`, `PemindahanAsetDetail`



Gambar: Pemindahan Aset

Pemindahan aset **tidak** mencatat realisasi RKAT (bukan pengeluaran).

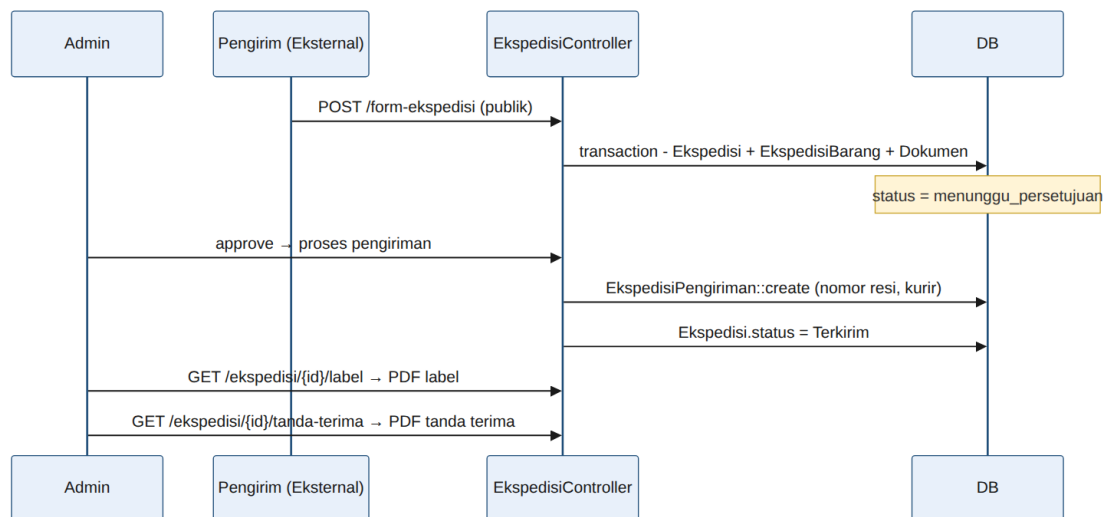
4.8 Ekspedisi (Pengiriman Keluar)

Dalam bahasa sederhana

1. Pengirim mengisi form pengiriman barang/dokumen.
2. Admin menyetujui, lalu mencatat nomor resi dan kurir. Status menjadi "Terkirim".
3. Admin bisa mencetak label pengiriman dan tanda terima dalam bentuk PDF.

Controller: EkspedisiController

Model: Ekspedisi, EkspedisiBarang, EkspedisiDokumen, EkspedisiPengiriman



Gambar: Ekspedisi (Pengiriman Keluar)

4.9 Pengaduan Kerusakan

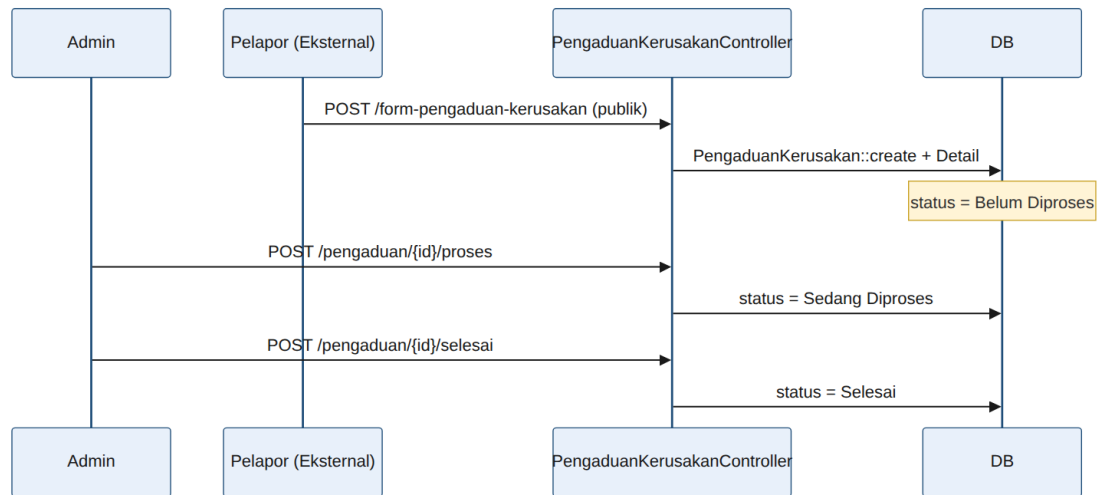
Dalam bahasa sederhana

1. Siapa pun bisa melaporkan kerusakan lewat form (tanpa login).
2. Admin menandai laporan "Sedang Diproses" saat mulai ditangani.
3. Setelah beres, laporan ditandai "Selesai".

Controller: PengaduanKerusakanController

Model: PengaduanKerusakan, PengaduanKerusakanDetail

Form publik tanpa login. Admin menangani laporan dan menautkannya ke Maintenance jika perlu.



Gambar: Pengaduan Kerusakan

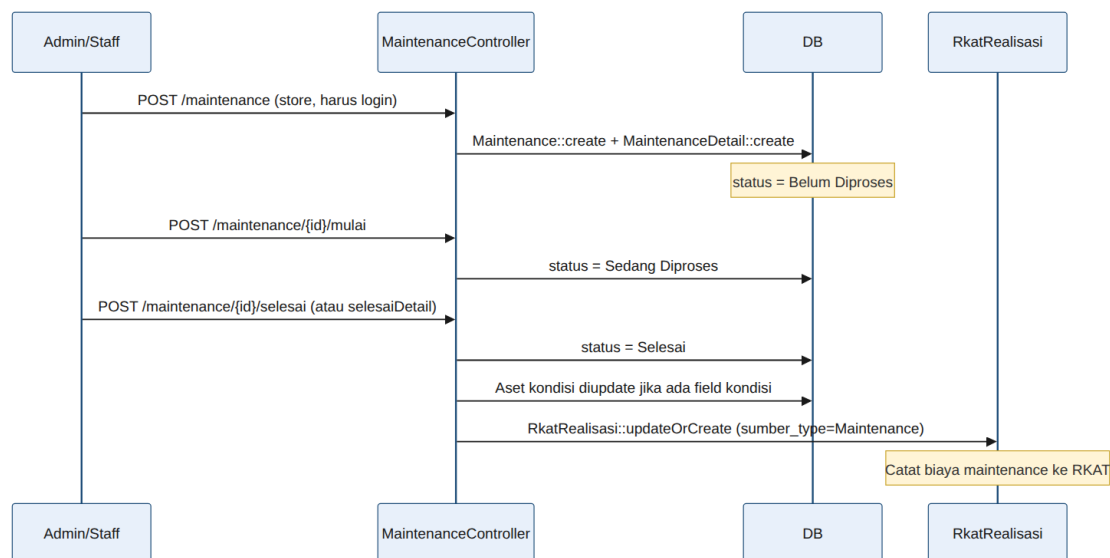
4.10 Maintenance

Dalam bahasa sederhana

1. Staf membuat catatan perawatan/perbaikan aset.
2. Saat mulai dikerjakan, status menjadi "Sedang Diproses".
3. Saat selesai, kondisi aset diperbarui dan biaya perawatan otomatis tercatat ke anggaran (RKAT).

Controller: MaintenanceController

Model: Maintenance, MaintenanceDetail



Gambar: Maintenance

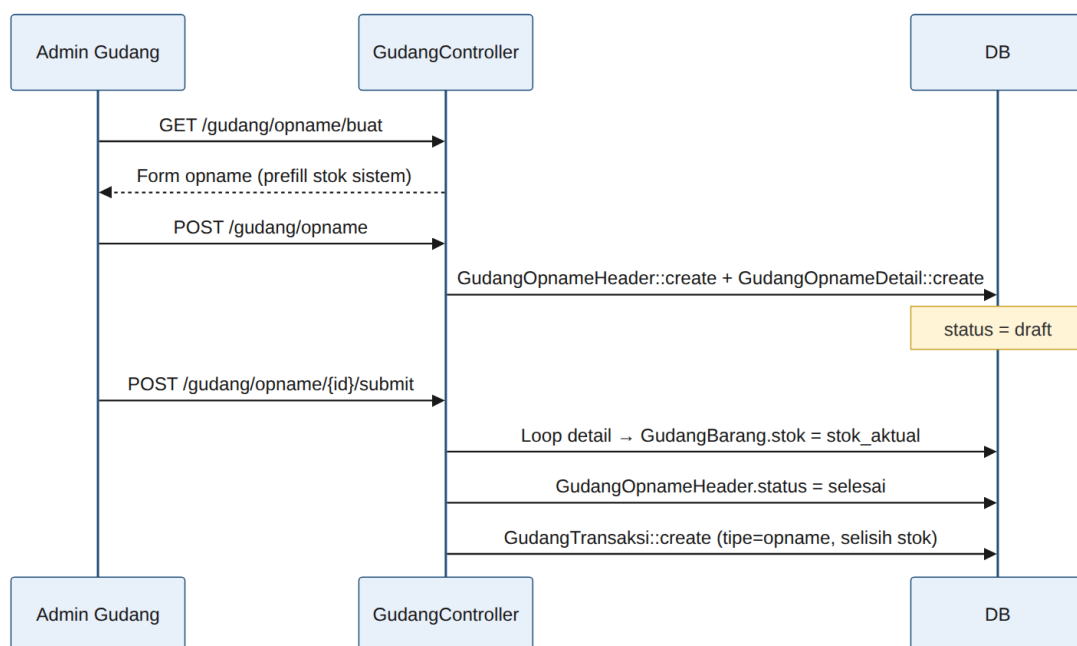
4.11 Stok Opname Gudang

Dalam bahasa sederhana

1. Stok opname adalah menghitung jumlah barang secara fisik lalu mencocokkannya dengan catatan sistem.
2. Admin membuat draft opname (angka dari sistem sudah terisi otomatis), lalu mengisi hasil hitungan nyata.
3. Saat disubmit, stok di sistem disamakan dengan hasil hitungan nyata dan selisihnya dicatat.

Controller: `GudangController` — method `opname*()`

Model: `GudangOpnameHeader`, `GudangOpnameDetail`



Gambar: Stok Opname Gudang

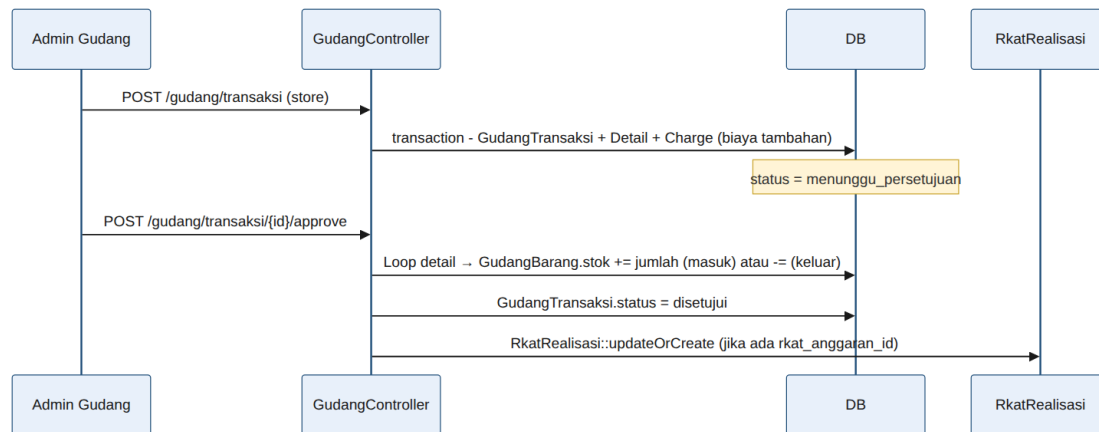
4.12 Transaksi Gudang (Masuk / Keluar)

Dalam bahasa sederhana

1. Admin gudang mencatat barang masuk atau keluar, termasuk biaya tambahan (misalnya ongkos kirim).
2. Transaksi harus disetujui dulu.
3. Setelah disetujui, stok otomatis bertambah/berkurang, dan biayanya masuk anggaran jika terkait pos RKAT.

Controller: `GudangController` — method `transaksiStore()`, `transaksiApprove()`

Model: `GudangTransaksi`, `GudangTransaksiDetail`, `GudangTransaksiCharge`



Gambar: Transaksi Gudang (Masuk / Keluar)

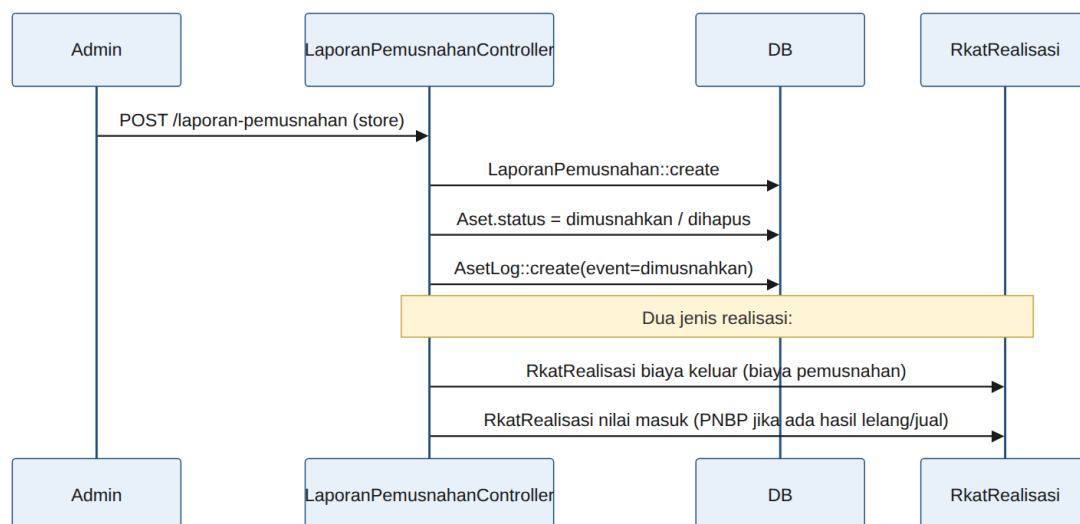
4.13 Laporan Pemusnahan Aset

Dalam bahasa sederhana

1. Admin mencatat aset yang dimusnahkan atau dijual/dilelang.
2. Status aset otomatis berubah menjadi "dimusnahkan" dan riwayatnya dicatat.
3. Biaya pemusnahan dicatat sebagai pengeluaran anggaran. Jika ada hasil penjualan, dicatat sebagai pemasukan (PNBP).

Controller: LaporanPemusnahanController

Model: LaporanPemusnahan



Gambar: Laporan Pemusnahan Aset

4.14 Rekap Bulanan Gudang (Scheduled Task)

Dalam bahasa sederhana

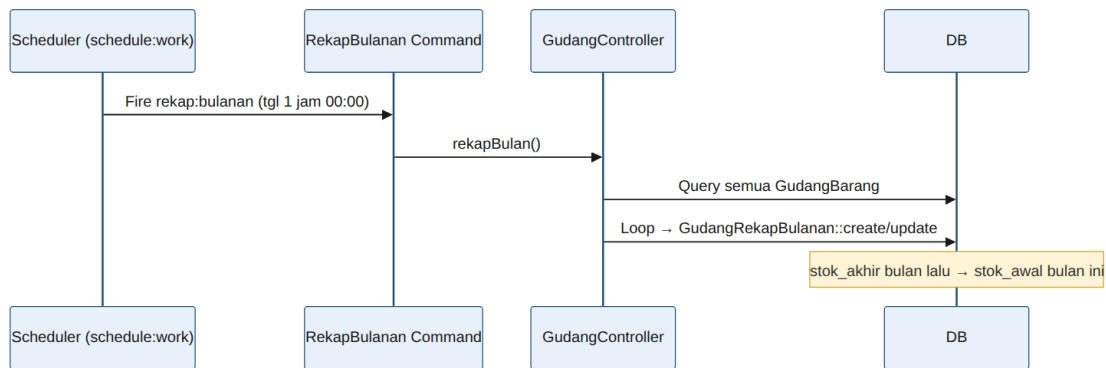
1. Setiap tanggal 1 pukul 00:00, sistem otomatis "memotret" stok akhir semua barang gudang.

2. Stok akhir bulan lalu menjadi stok awal bulan ini.
3. Jika proses otomatis terlewat, bisa dijalankan manual (lihat SOP-09 di Bab 9).

Command: `php artisan rekap:bulanan`

Controller: `GudangController::rekapBulan()`

Jadwal: Otomatis tiap tanggal 1 jam 00:00



Gambar: Rekap Bulanan Gudang (Scheduled Task)

Bisa juga dipicu manual: `php artisan rekap:bulanan`.

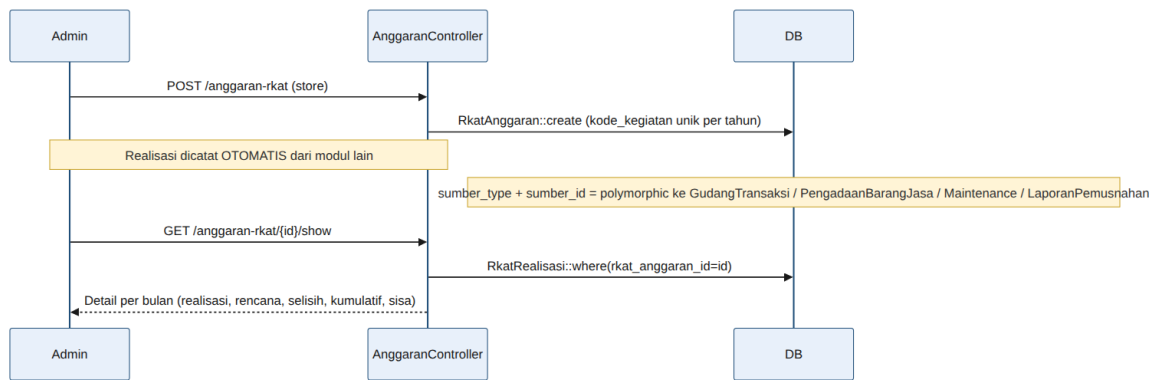
4.15 Anggaran RKAT

Dalam bahasa sederhana

1. Admin memasukkan rencana anggaran tahunan per kegiatan, termasuk rincian rencana per bulan.
2. Pemakaian anggaran (realisasi) TIDAK diinput manual. Realisasi tercatat otomatis setiap kali ada pengadaan, maintenance, transaksi gudang, atau pemusnahan yang selesai diproses.
3. Admin bisa melihat perbandingan rencana dan realisasi per bulan, termasuk sisa anggaran.

Controller: `AnggaranController`

Model: `RkatAnggaran`, `RkatRealisasi`



Gambar: Anggaran RKAT

Realisasi **tidak dicatat manual oleh admin** — dicatat otomatis saat modul pengadaan/maintenance/gudang/pemusnahan diproses.

4.16 Form Publik — Keamanan

Penjelasan sederhana

Form publik bisa diakses siapa saja tanpa login, sehingga perlu beberapa lapis pengaman: pembatasan jumlah kiriman (maksimal 20 per menit dari satu alamat internet), token anti-pemalsuan (CSRF), pemeriksaan isi form, dan pembatasan jenis serta ukuran file. Yang BELUM ada adalah CAPTCHA (tes "saya bukan robot").

Semua form publik (tanpa login) dilindungi:

Mekanisme	Implementasi
Rate limiting	<code>throttle:20,1</code> pada route POST publik — max 20 request/menit per IP
CSRF	Laravel default CSRF token di semua form
Validasi input	<code>\$request->validate()</code> di controller — wajib sebelum insert DB
Upload validation	<code>mimes:jpg,jpeg,png,pdf + max:2048</code> (2MB) untuk file upload
SQL Injection	Eloquent ORM / parameter binding — tidak ada query raw tanpa binding
XSS	Blade <code>{{ }}</code> auto-escape; <code>{!! !!}</code> hanya untuk HTML yang sudah disanitasi

Catatan

Standar yang ditetapkan: semua formulir publik wajib memakai Google reCAPTCHA v3 (tes "bukan robot" yang tidak terlihat oleh pengguna). Saat dokumen ini dibuat, CAPTCHA belum terpasang di kode dan masuk daftar pekerjaan — lihat **Bab 8 (Risiko)** poin A6 dan R1.

ALUR DATA & STRUKTUR DATABASE

SINGKATNYA

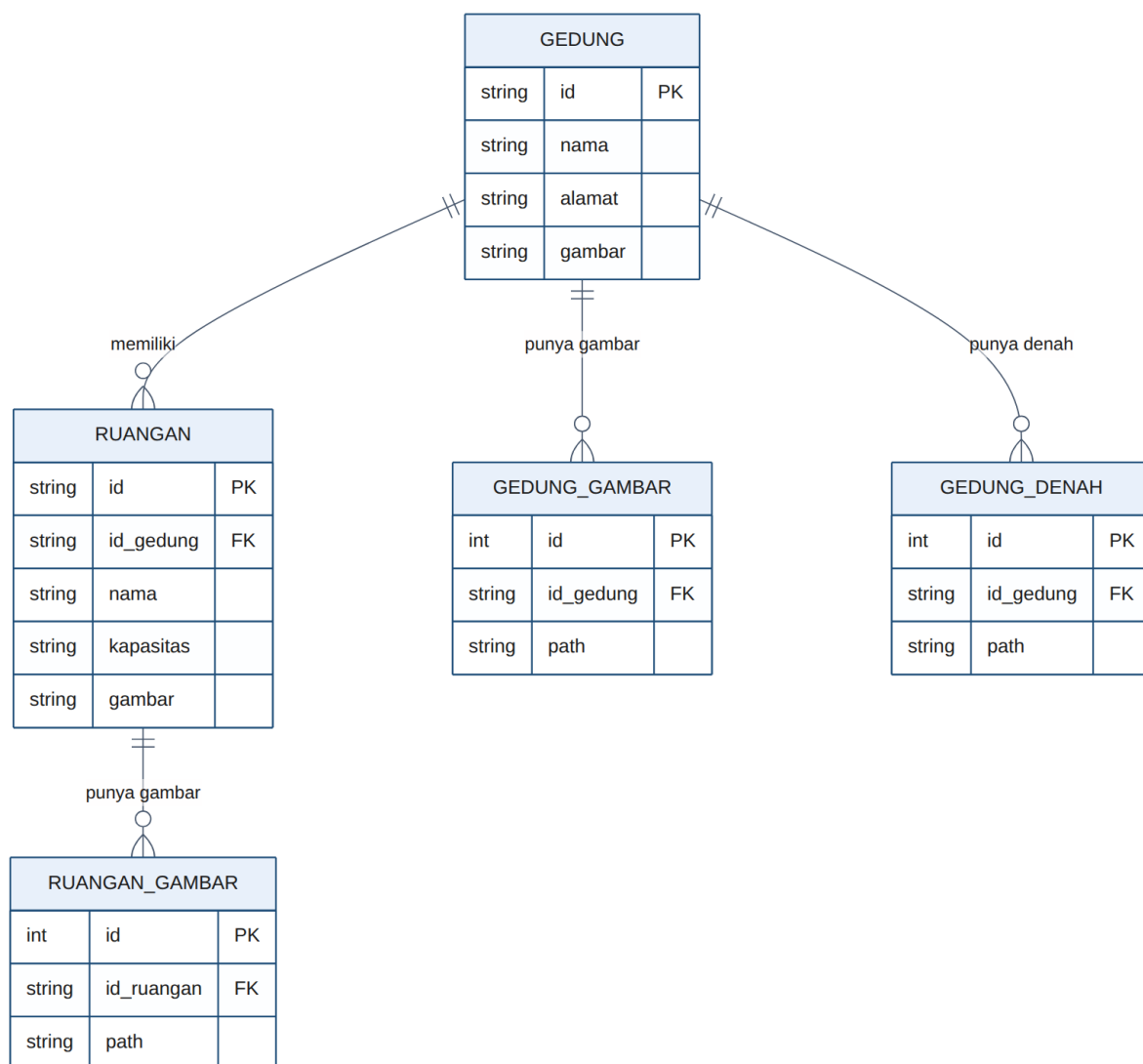
1. Database bisa dibayangkan sebagai kumpulan tabel Excel yang saling terhubung. ERD (Entity Relationship Diagram) adalah peta yang menunjukkan tabel apa saja yang ada dan bagaimana hubungannya.
2. Cara membaca ERD: setiap kotak adalah satu tabel, baris di dalamnya adalah kolom. PK (Primary Key) adalah nomor identitas unik setiap baris, seperti NIK. FK (Foreign Key) adalah kolom yang "menunjuk" ke tabel lain. Garis dengan tanda satu-ke-banyak artinya satu data bisa punya banyak data terkait, misalnya satu gedung punya banyak ruangan.
3. Endpoint adalah "alamat" di aplikasi yang bisa dipanggil. GET = membuka atau mengambil data, POST = mengirim atau menyimpan data baru, PUT = mengubah data, DELETE = menghapus data.

5.1 ERD per Modul

Penjelasan sederhana

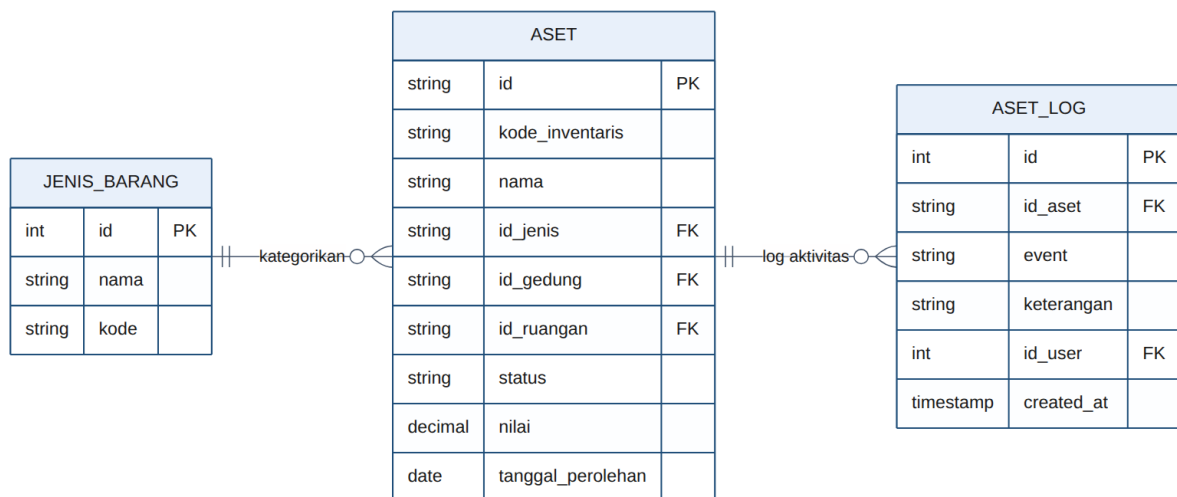
Di bawah ini setiap modul punya peta tabelnya sendiri, supaya tidak menumpuk menjadi satu gambar raksasa yang sulit dibaca.

5.1.1 Modul Infrastruktur



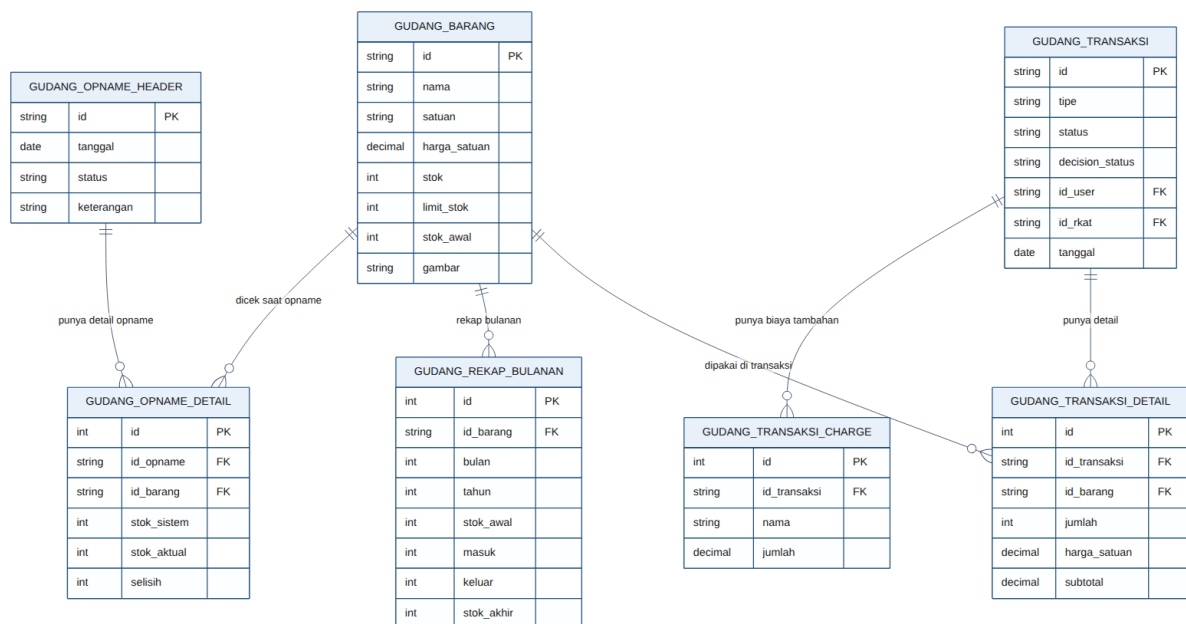
Gambar: Modul Infrastruktur

5.1.2 Modul Aset Inventaris



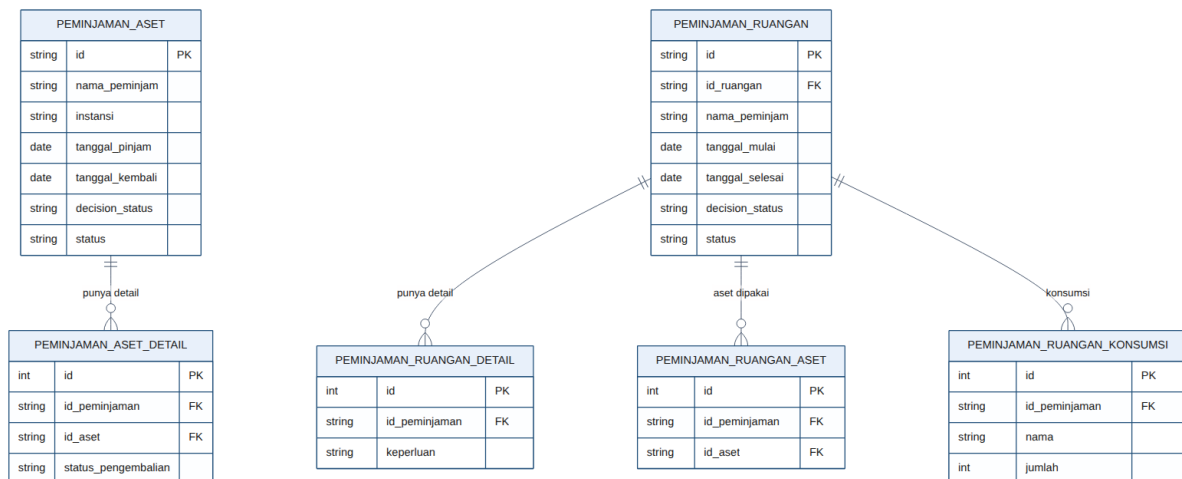
Gambar: Modul Aset Inventaris

5.1.3 Modul Gudang



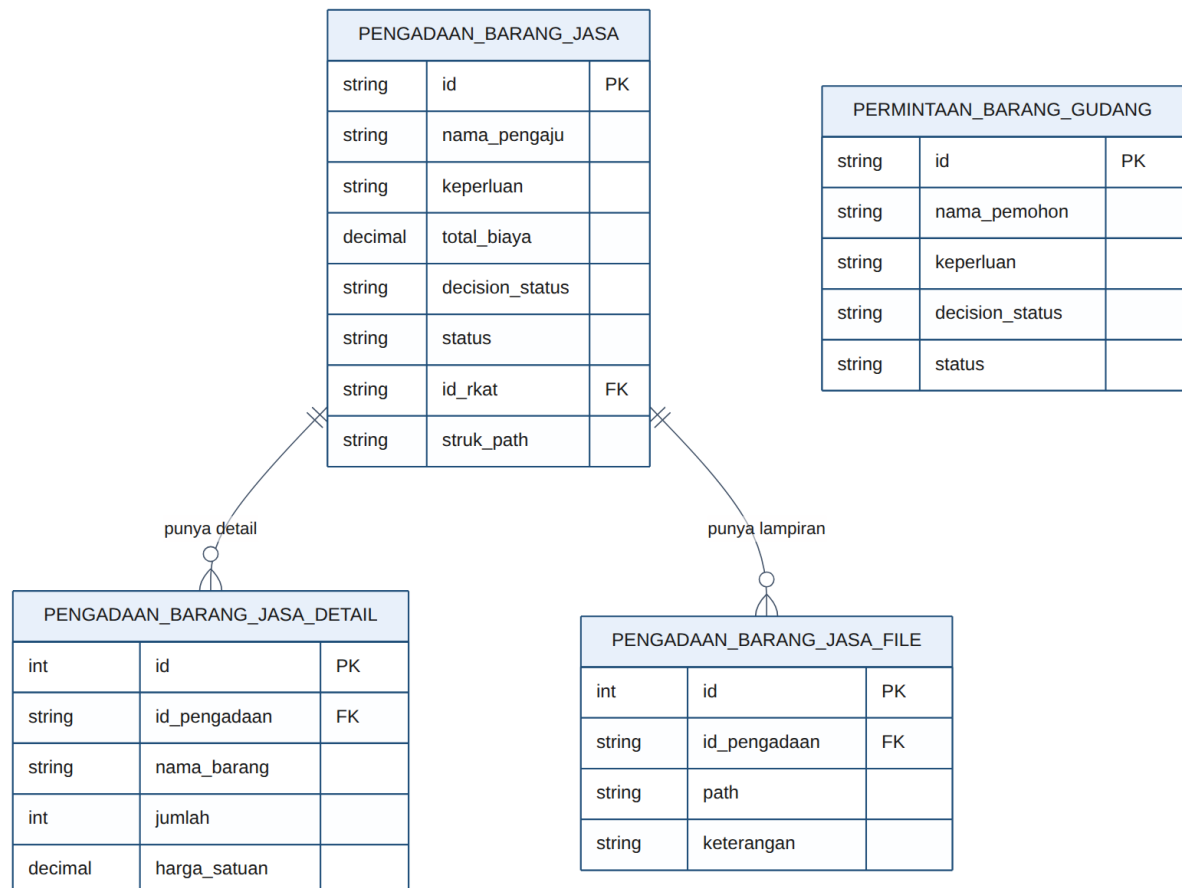
Gambar: Modul Gudang

5.1.4 Modul Peminjaman



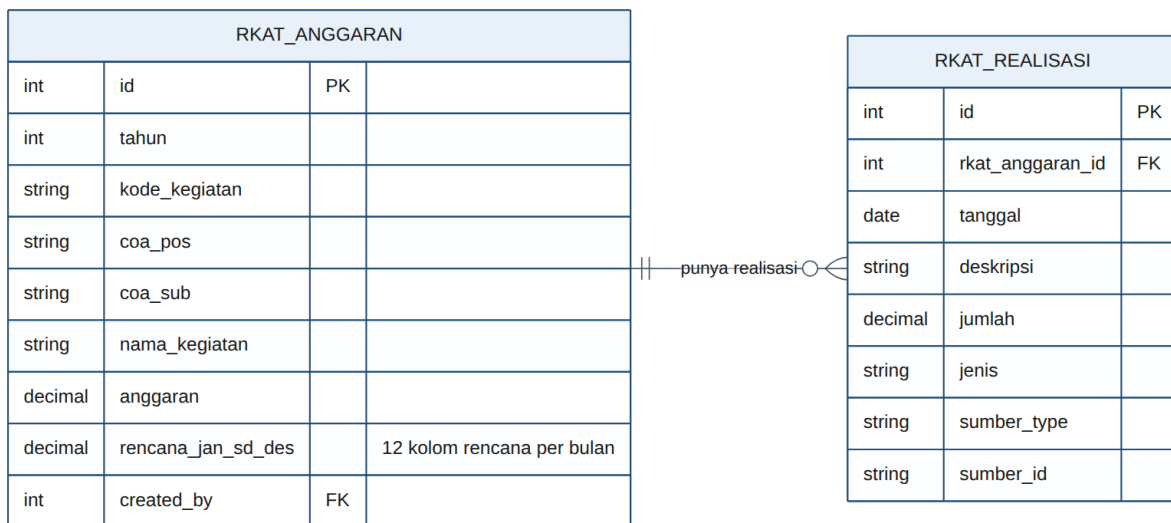
Gambar: Modul Peminjaman

5.1.5 Modul Pengadaan & Permintaan



Gambar: Modul Pengadaan & Permintaan

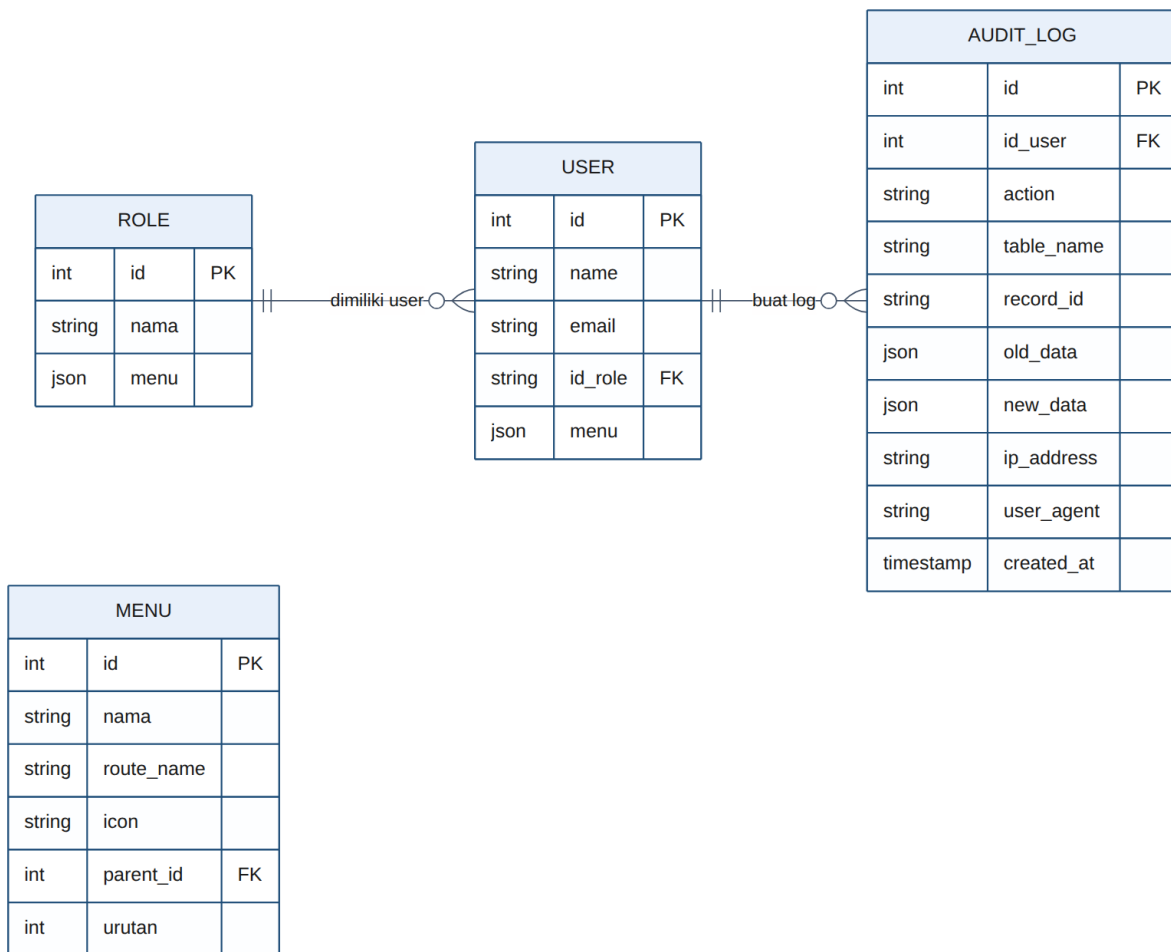
5.1.6 Modul RKAT Anggaran



Gambar: Modul RKAT Anggaran

`sumber_type` + `sumber_id` adalah **polymorphic key** — menunjuk ke record sumber di tabel lain (contoh: `sumber_type = App\Models\GudangTransaksi`, `sumber_id = TRX-20260901-0001`).

5.1.7 Modul RBAC & User



Gambar: Modul RBAC & User

5.2 Daftar Endpoint per Modul

Penjelasan sederhana

Daftar ini adalah semua "alamat" yang bisa diakses di aplikasi, dikelompokkan per modul. Yang bertanda PUBLIK bisa dibuka tanpa login. AJAX artinya dipanggil otomatis oleh halaman di belakang layar (misalnya saat mengecek ketersediaan).

Format: [METHOD] /path → Controller::method()

5.2.1 Infrastruktur

GET /gedung	→ GedungController::index
POST /gedung	→ GedungController::store
GET /gedung/{id}/edit	→ GedungController::edit
PUT /gedung/{id}	→ GedungController::update
DELETE /gedung/{id}	→ GedungController::destroy
GET /ruangan	→ RuanganController::index
POST /ruangan	→ RuanganController::store

5.2.2 Aset Inventaris

GET /aset	→ AsetController::index
POST /aset	→ AsetController::store
GET /aset/{id}	→ AsetController::show
PUT /aset/{id}	→ AsetController::update
DELETE /aset/{id}	→ AsetController::destroy
GET /aset/export-excel	→ AsetController::exportExcel
GET /aset/export-pdf	→ AsetController::exportPdf

5.2.3 Gudang

GET /gudang	→ GudangController::index
POST /gudang	→ GudangController::store (tambah barang)
PUT /gudang/{id}	→ GudangController::update
DELETE /gudang/{id}	→ GudangController::destroy
GET /gudang/transaksi	→ GudangController::transaksiIndex
POST /gudang/transaksi	→ GudangController::transaksiStore
GET /gudang/transaksi/{id}	→ GudangController::transaksiDetail
POST /gudang/transaksi/{id}/approve	→ GudangController::transaksiApprove
GET /gudang/opname	→ GudangController::opnameIndex
POST /gudang/opname	→ GudangController::opnameStore
POST /gudang/opname/{id}/submit	→ GudangController::opnameSubmit
GET /gudang/rekap	→ GudangController::rekapIndex

5.2.4 Peminjaman Aset

GET /form_peminjaman_aset	→ PeminjamanAsetController::create (PUBLIK)
POST /form_peminjaman_aset	→ PeminjamanAsetController::store (PUBLIK)
GET /cek-ketersediaan	→ PeminjamanAsetController::cekKetersediaan (AJAX)
GET /peminjaman-aset	→ PeminjamanAsetController::index
GET /peminjaman-aset/{id}	→ PeminjamanAsetController::show
POST /peminjaman-aset/{id}/approve	→ PeminjamanAsetController::approve
POST /peminjaman-aset/{id}/reject	→ PeminjamanAsetController::reject
POST /peminjaman-aset/{id}/serahkan	→ PeminjamanAsetController::serahkanItem
POST /peminjaman-aset/{id}/kembalikan	→ PeminjamanAsetController::kembalikan
GET /peminjaman-aset/export-pdf	→ PeminjamanAsetController::exportPdf

5.2.5 Peminjaman Ruangan

GET /form-peminjaman-ruangan	→ PeminjamanRuanganController::create (PUBLIK)
POST /form-peminjaman-ruangan	→ PeminjamanRuanganController::store (PUBLIK)
GET /cek-ketersediaan-ruangan	→ PeminjamanRuanganController::cekKetersediaanRuangan (AJAX)
GET /peminjaman-ruangan	→ PeminjamanRuanganController::index
POST /peminjaman-ruangan/{id}/approve	→ PeminjamanRuanganController::approve
POST /peminjaman-ruangan/{id}/reject	→ PeminjamanRuanganController::reject

5.2.6 Permintaan & Pengadaan

GET	/form-permintaan-barang	→ PermintaanBarangGudangController::create (PUBLIK)
POST	/form-permintaan-barang	→ PermintaanBarangGudangController::store (PUBLIK)
GET	/permintaan-barang	→ PermintaanBarangGudangController::index
POST	/permintaan-barang/{id}/approve	→ PermintaanBarangGudangController::approve
POST	/permintaan-barang/{id}/proses	→ PermintaanBarangGudangController::proses
GET	/form-pengadaan-barang	→ PengadaanBarangJasaController::create (PUBLIK)
POST	/form-pengadaan-barang	→ PengadaanBarangJasaController::store (PUBLIK)
GET	/pengadaan-barang	→ PengadaanBarangJasaController::index
GET	/pengadaan-barang/{id}	→ PengadaanBarangJasaController::show
POST	/pengadaan-barang/{id}/approve	→ PengadaanBarangJasaController::approve
POST	/pengadaan-barang/{id}/selesai	→ PengadaanBarangJasaController::selesai

5.2.7 Permintaan Kendaraan

GET	/form-permintaan-kendaraan	→ PermintaanKendaraanController::create (PUBLIK)
POST	/form-permintaan-kendaraan	→ PermintaanKendaraanController::store (PUBLIK)
GET	/cek-kendaraan	→ PermintaanKendaraanController::cekKetersediaan (AJAX)
GET	/permintaan-kendaraan	→ PermintaanKendaraanController::index
POST	/permintaan-kendaraan/{id}/approve	→ PermintaanKendaraanController::approve

5.2.8 Maintenance

GET	/maintenance	→ MaintenanceController::index
POST	/maintenance	→ MaintenanceController::store
GET	/maintenance/{id}	→ MaintenanceController::show
POST	/maintenance/{id}/mulai	→ MaintenanceController::mulai
POST	/maintenance/{id}/selesai	→ MaintenanceController::selesai
POST	/maintenance/{id}/detail/{id}/selesai	→ MaintenanceController::selesaiDetail

5.2.9 Anggaran RKAT

GET	/anggaran-rkat	→ AnggaranController::index
POST	/anggaran-rkat	→ AnggaranController::store
PUT	/anggaran-rkat/{id}	→ AnggaranController::update
DELETE	/anggaran-rkat/{id}	→ AnggaranController::destroy
GET	/anggaran-rkat/{id}/show	→ AnggaranController::showRkat
GET	/riwayat-realisisasi	→ AnggaranController::indexRealisasi
POST	/riwayat-realisisasi	→ AnggaranController::storeRealisasi
DELETE	/riwayat-realisisasi/{id}	→ AnggaranController::destroyRealisasi

5.2.10 Admin

GET	/users	→ UserController::index
POST	/users	→ UserController::store
PUT	/users/{id}	→ UserController::update
DELETE	/users/{id}	→ UserController::destroy
GET	/roles	→ RolesController::index
POST	/roles	→ RolesController::store
PUT	/roles/{id}	→ RolesController::update
GET	/menus	→ MenuController::index
POST	/menus	→ MenuController::store
GET	/audit-log	→ AuditLogController::index

5.2.11 Vendor (Publik)

```
GET /vendor/create → VendorController::create (PUBLIK)
POST /vendor → VendorController::store (PUBLIK)
```

5.3 Format ID (Primary Key)

Penjelasan sederhana

Setiap transaksi punya nomor unik seperti nomor resi. Contoh: PJM-20260901-0001 artinya Peminjaman Aset (PJM), tanggal 1 September 2026, urutan pertama pada hari itu.

Hampir semua tabel transaksional menggunakan string PK format **PREFIX-YYYYMMDD-NNNN**:

Prefix	Modul	Contoh
PJM	Peminjaman Aset	PJM-20260901-0001
PMR	Peminjaman Ruangan	PMR-20260901-0001
TRX	Transaksi Gudang	TRX-20260901-0001
EKS	Ekspedisi	EKS-20260901-0001
MNT	Maintenance	MNT-20260901-0001
PMD	Pemindahan Aset	PMD-20260901-0001
PNG	Pengadaan Barang/Jasa	PNG-20260901-0001
PKD	Permintaan Kendaraan	PKD-20260901-0001

Generator ada di `generateId()` di masing-masing controller. Pakai `lockForUpdate()` saat store untuk menghindari race condition (lihat **Bab 8 (Risiko)** untuk analisis lengkap).

5.4 Data Flow: Realisasi RKAT Otomatis

Penjelasan sederhana

Artinya: setiap kali ada uang keluar atau masuk dari empat modul di bawah ini, sistem otomatis mencatatnya ke anggaran. Jika transaksi yang sama diproses ulang, catatannya diperbarui, bukan digandakan.

Realisasi RKAT dicatat dari 4 modul sumber:

```
GudangTransaksi::approve()
    → RkatRealisasi::updateOrCreate(sumber_type=GudangTransaksi, sumber_id=TRX-...)

PengadaanBarangJasa::selesai()
    → RkatRealisasi::updateOrCreate(sumber_type=PengadaanBarangJasa, sumber_id=PNG-...)

MaintenanceController::selesai() / selesaiDetail()
    → RkatRealisasi::updateOrCreate(sumber_type=Maintenance, sumber_id=MNT-...)

LaporanPemusnahanController::store()
    → RkatRealisasi biaya keluar (sumber_type=LaporanPemusnahan)
    → RkatRealisasi nilai masuk / PNBP (sumber_type=LaporanPemusnahan, jenis=masuk)
```

`updateOrCreate` memastikan jika transaksi yang sama di-reprocess, record realisasi diupdate (tidak duplikat).

5.5 Audit Log — Semua Perubahan Data

Penjelasan sederhana

Ini isi "rekaman CCTV" data. Setiap perubahan menyimpan siapa pelakunya, jenis aksinya, tabel mana yang berubah, data lama dan data baru, serta alamat IP dan browser yang dipakai.

Setiap insert/update/delete di model yang extend `BaseModel` otomatis tercatat di `audit_logs`:

Field	Isi
<code>id_user</code>	User yang login saat itu (null jika dari console/queue)
<code>action</code>	<code>create</code> / <code>update</code> / <code>delete</code>
<code>table_name</code>	Nama tabel yang berubah
<code>record_id</code>	Primary key record yang berubah
<code>old_data</code>	JSON snapshot sebelum perubahan (null untuk create)
<code>new_data</code>	JSON snapshot setelah perubahan (null untuk delete)
<code>ip_address</code>	IP client
<code>user_agent</code>	Browser/client string

`AuditLog` model sendiri **tidak** extend `BaseModel` — untuk menghindari infinite loop.

INSTALASI, DEPLOYMENT & BACKUP

SINGKATNYA

1. Bab ini berisi cara memasang aplikasi dari nol dan cara memperbaruinya di server. Ada dua lingkungan: Development (lingkungan uji coba di laptop programmer, datanya boleh rusak) dan Production (sistem asli yang dipakai sehari-hari, datanya tidak boleh hilang).
2. Bagian terpenting untuk semua anggota tim adalah Backup & Restore. Saat ini sistem BELUM punya backup otomatis, jadi bagian itu wajib dibaca.

6.1 Setup Lokal (Development)

Penjelasan sederhana

Bagian ini untuk programmer yang ingin menjalankan aplikasi di laptopnya sendiri untuk uji coba. Tidak berpengaruh ke sistem asli.

6.1.1 Prasyarat

- PHP ≥ 8.2 (dengan ekstensi: `pdo_mysql`, `gd`, `zip`, `bcmath`, `intl`, `mbstring`, `xml`, `dom`)
- Composer $\geq 2.x$
- Node.js $\geq 22.x$ + npm
- SQLite (bawaan PHP) atau MySQL 8.0 (opsional)

6.1.2 Langkah Setup Pertama Kali

```
# 1. Clone repo
git clone https://github.com/pinpinpinaaaaa/manajemen-aset.git
cd manajemen-aset

# 2. Jalankan setup lengkap (install deps + env + key + migrate + build assets)
composer run setup
# Ekuivalen dengan:
# composer install
# cp .env.example .env
# php artisan key:generate
# php artisan migrate
# npm install && npm run build

# 3. Buat symlink storage (untuk akses file upload via browser)
php artisan storage:link

# 4. Jalankan dev environment (server + queue + log tail + vite)
composer run dev
```

Aplikasi berjalan di: `http://localhost:8000`

6.1.3 Akun Default (dari Seeder)

Seed data awal dijalankan dengan:

```
php artisan db:seed
# PERHATIAN: db:seed berbeda dari db:seed DatabaseSeeder — seeder aman, tidak menghapus data
```

Perhatian

JANGAN jalankan `migrate:fresh`, `db:wipe`, atau `migrate:fresh --seed` di environment yang sudah ada data — ini akan **menghapus semua data**.

6.2 Environment Variables (.env)

Penjelasan sederhana

File `.env` adalah "buku catatan pengaturan rahasia" aplikasi: alamat database, password, kunci enkripsi, dan sebagainya. File ini TIDAK BOLEH dibagikan, dikirim lewat chat, atau diunggah ke GitHub.

Variable	Default	Keterangan
APP_NAME	"Manajemen Aset"	Nama aplikasi, muncul di tab browser
APP_ENV	local	local / production — affects error display
APP_KEY	(generated)	Encryption key — JANGAN share / commit
APP_DEBUG	true	Set false di production
APP_URL	http://localhost:8000	URL publik aplikasi
APP_PORT	8000	Port Nginx (Docker)
DB_CONNECTION	sqlite	sqlite atau mysql
DB_HOST	127.0.0.1	Host MySQL (mysql untuk Docker dev, 127.0.0.1 untuk prod via CF tunnel)
DB_PORT	3306	Port MySQL
DB_DATABASE	manajemen_aset	Nama database
DB_USERNAME	root	Username database
DB_PASSWORD	—	Password database — JANGAN commit
SESSION_DRIVER	database	Driver session — wajib migrate sebelum pakai
QUEUE_CONNECTION	database	Driver queue
CACHE_STORE	database	Driver cache
MAIL_MAILER	log	log = tidak kirim email; set smtp untuk email aktif
MAIL_HOST	—	SMTP host (jika MAIL_MAILER=smtp)
MAIL_PORT	—	SMTP port
MAIL_USERNAME	—	SMTP user
MAIL_PASSWORD	—	SMTP password

Variable	Default	Keterangan
CF_ACCESS_HOSTNAME	—	Hostname Cloudflare tunnel ke database (prod)
CF_ACCESS_CLIENT_ID	—	Client ID Cloudflare Zero Trust (prod)
CF_ACCESS_CLIENT_SECRET	—	Client Secret Cloudflare Zero Trust (prod)
GITHUB_USER	—	Username GitHub untuk pull image dari GHCR (opsional, prod)

6.3 Docker (Development)

Penjelasan sederhana

Docker membungkus aplikasi ke dalam "kotak-kotak" (container) sehingga semua orang menjalankan aplikasi dengan lingkungan yang sama persis. Perintah di bawah ini dipakai untuk menyalakan, mematikan, dan melihat catatan kotak-kotak tersebut.

6.3.1 Struktur Container

```
docker-compose.yml (dev):
├── app      — PHP-FPM (Laravel app)
├── nginx    — Nginx web server (port 8000:80)
├── mysql    — MySQL 8.0 (port 3306:3306)
├── worker   — Queue worker (queue:listen)
└── scheduler — Scheduler (schedule:work)
```

6.3.2 Perintah Docker Dev

```
# Jalankan semua container
docker compose up -d

# Lihat log
docker compose logs -f app
docker compose logs -f worker

# Masuk ke container app
docker compose exec app bash

# Jalankan artisan dari luar container
docker compose exec app php artisan migrate
docker compose exec app php artisan rekap:bulanan

# Stop semua container
docker compose down
```

6.3.3 Build Image

```
# Build image dari Dockerfile (multi-stage)
docker build -t manajemen-aset:latest .

# Build untuk production (tanpa dev deps)
docker build --target nginx-stage -t manajemen-aset-nginx:latest .
```

6.4 Docker (Production)

Penjelasan sederhana

Production adalah server sistem asli. Di production, database tidak berada di dalam Docker, melainkan di server kantor (on-premise) yang dihubungkan lewat Cloudflare Tunnel.

6.4.1 Struktur Container Prod

```
docker-compose.prod.yml:
├── app      — PHP-FPM (image dari GHCR atau build lokal)
├── nginx    — Nginx
├── worker   — Queue worker
└── scheduler — Scheduler
# Tidak ada container MySQL — DB di server on-premise via Cloudflare tunnel
```

6.4.2 Deploy ke Production

```
# Pull image terbaru
docker compose -f docker-compose.prod.yml pull

# Jalankan (recreate jika image berubah)
docker compose -f docker-compose.prod.yml up -d --force-recreate

# Lihat status
docker compose -f docker-compose.prod.yml ps
```

6.4.3 Entrypoint Container (docker/entrypoint.sh)

Saat container `app` start, script ini dijalankan secara berurutan:

1. Cek `CF_ACCESS_CLIENT_ID` → jika ada, jalankan cloudflared (tunnel ke DB)
2. Tunggu koneksi database siap (retry tiap 2 detik)
3. `php artisan storage:link`
4. `php artisan migrate --force` (jalankan migration baru)
5. `php artisan config:cache`
6. `php artisan route:cache`
7. `php artisan view:cache`
8. `php-fpm`

Perhatian

Penting: `migrate --force` dijalankan otomatis saat container start di production. Pastikan migration baru bersifat **additive** (tidak DROP TABLE atau DROP COLUMN) sebelum deploy.

6.5 Cloudflare Zero Trust Tunnel

Penjelasan sederhana

Bayangkan jalan tol pribadi berpagar yang hanya bisa dilewati kendaraan berkartu khusus. Aplikasi memakai "kartu" (Client ID & Client Secret) untuk masuk ke jalur ini, lalu sampai ke database di kantor. Keuntungannya: database tidak perlu dibuka ke internet sehingga lebih aman. Jika kartunya kedaluwarsa, aplikasi tidak bisa mengakses data.

Dipakai di **production** untuk koneksi dari Docker container ke database MySQL on-premise.

6.5.1 Cara Kerja

```
Container app/worker/scheduler
↓ connect ke DB_HOST=127.0.0.1:3306
cloudflared (berjalan di background dalam container)
↓ TCP proxy tunnel
Cloudflare Zero Trust Network
↓ meneruskan ke
MySQL on-premise server
```

6.5.2 Setup Cloudflare Tunnel

1. Login ke Cloudflare Zero Trust dashboard
2. Buat **Application** bertipe TCP
3. Buat **Service Token** → dapat **CF_ACCESS_CLIENT_ID** dan **CF_ACCESS_CLIENT_SECRET**
4. Atur **hostname** untuk tunnel → **CF_ACCESS_HOSTNAME**
5. Set ketiga variable tersebut di **.env** production

6.5.3 Konfigurasi .env untuk Prod

```
CF_ACCESS_HOSTNAME=db.yourdomain.com
CF_ACCESS_CLIENT_ID=xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx.access
CF_ACCESS_CLIENT_SECRET=xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
DB_HOST=127.0.0.1
DB_PORT=3306
```

Catatan

Jika **CF_ACCESS_CLIENT_ID** tidak diset di **.env**, cloudflared **tidak dijalankan** dan koneksi DB langsung ke **DB_HOST** (cocok untuk dev dengan MySQL container lokal).

6.6 Backup & Restore

Penjelasan sederhana

Backup adalah membuat salinan cadangan data. Restore adalah mengembalikan data dari salinan cadangan tersebut. Tanpa backup, jika server rusak atau data terhapus tidak sengaja, SEMUA DATA HILANG PERMANEN. Saat ini backup masih harus dilakukan manual.

Catatan

Standar yang ditetapkan: backup database **otomatis setiap hari pukul 02:00 WIB**, file upload **setiap minggu (Minggu pukul 03:00 WIB)**, disimpan **30 hari di 2 lokasi** (disk backup server database + NAS/cloud). Saat dokumen ini dibuat, backup otomatis belum terpasang — pasang dengan langkah di subbab "Rekomendasi Backup Otomatis" di bawah. Lihat juga **Bab 8 (Risiko)** poin A4.

6.6.1 Backup Manual Database

```
# Backup dari container MySQL (dev)
docker compose exec mysql mysqldump -u root -p manajemen_aset > backup_$(date +%Y%m%d).sql

# Backup dari server MySQL eksternal (prod — jalankan dari server DB)
mysqldump -u USERNAME -p manajemen_aset > backup_$(date +%Y%m%d_%H%M%S).sql
```

6.6.2 Backup File Upload (storage/)

```
# Dari host Docker, backup named volume
docker run --rm \
-v manajemen-aset_storage_data:/data \
-v $(pwd):/backup \
alpine tar czf /backup/storage_backup_$(date +%Y%m%d).tar.gz /data
```

6.6.3 Restore Database

```
# Restore ke MySQL (hati-hati: ini menimpa data yang ada)
mysql -u USERNAME -p manajemen_aset < backup_20260901.sql

# Atau dari container:
docker compose exec -T mysql mysql -u root -p manajemen_aset < backup_20260901.sql
```

6.6.4 Restore File Upload

```
# Extract backup ke volume
docker run --rm \
-v manajemen-aset_storage_data:/data \
-v $(pwd):/backup \
alpine tar xzf /backup/storage_backup_20260901.tar.gz -C /
```

6.6.5 Rekomendasi Backup Otomatis

Jadwalkan di cron server (bukan di dalam Laravel scheduler karena scheduler ada di container):

```
# /etc/cron.d/manajemen-aset-backup
0 2 * * * root mysqldump -u backup_user -pPASSWORD manajemen_aset | gzip > /backups/db_$(date
+%\Y%\m%\d).sql.gz
# Simpan 30 hari terakhir:
0 3 * * * root find /backups -name "db_*.sql.gz" -mtime +30 -delete
```

6.7 Migration Policy

Penjelasan sederhana

Migration adalah "resep perubahan struktur database" (misalnya menambah kolom baru). Aturannya: migration hanya boleh MENAMBAH, tidak boleh menghapus tabel atau kolom, karena setiap kali aplikasi dinyalakan ulang migration dijalankan otomatis di sistem asli.

Wajib: semua migration bersifat **additive-only**:

Boleh (up())	JANGAN (up())
Schema::create(...)	Schema::dropIfExists(...)
\$table->addColumn(...)	\$table->dropColumn(...)
\$table->index(...)	Schema::drop(...)
\$table->foreign(...)	Modify column type yang bisa truncate data

Jika perlu hapus kolom/tabel lama, tandai dulu dengan `nullable()` → deploy → hapus di migration berikutnya setelah dipastikan tidak ada data yang bergantung.

Jalankan migration:

```
# Lokal
php artisan migrate

# Production (otomatis saat container start, atau manual):
docker compose exec app php artisan migrate --force
```

PANDUAN TROUBLESHOOTING

SINGKATNYA

1. Bab ini adalah "buku P3K" saat sistem bermasalah. Setiap masalah ditulis dengan pola yang sama: GEJALA (apa yang terlihat oleh pengguna), DIAGNOSA (cara mencari penyebabnya), dan SOLUSI (cara memperbaikinya).
2. Jika tidak tahu harus mulai dari mana, langsung lompat ke "Checklist Diagnosa Cepat" di akhir bab ini dan kerjakan dari atas ke bawah.

7.1 Arti Pesan Error yang Sering Muncul

Tabel ini membantu menebak jenis masalah hanya dari pesan yang terlihat di layar pengguna.

Pesan / gejala di layar	Artinya	Lihat bagian
403 Forbidden	Akses ditolak. User sudah login, tetapi jabatannya tidak diberi izin membuka halaman itu.	7.3.9
419 Page Expired	Halaman terlalu lama dibiarkan terbuka sehingga sesinya habis. Biasanya cukup refresh halaman lalu kirim ulang.	7.3.10
500 Internal Server Error	Ada error di dalam aplikasi. Penyebab pastinya harus dilihat di log.	7.3.1
502 Bad Gateway	Pintu depan (Nginx) hidup, tetapi "dapur" (aplikasi) mati atau tidak menjawab.	7.3.2
Connection refused / Access denied	Aplikasi tidak bisa terhubung ke database.	7.3.3 & 7.3.12
Gambar rusak (broken image)	File upload tidak bisa ditemukan oleh browser.	7.3.6

7.2 Lokasi Log

Penjelasan sederhana

Log adalah "buku harian" sistem: catatan otomatis tentang apa yang terjadi, termasuk semua error. Saat ada masalah, log adalah tempat pertama yang harus dilihat.

Log	Lokasi	Isi
Laravel app log	storage/logs/laravel.log	Error PHP, exception, query error, custom <code>Log::error()</code>
Nginx access log	stdout container <code>nginx</code>	Semua HTTP request + status code
Nginx error log	stderr container <code>nginx</code>	Nginx error (502, 504, config error)
MySQL log	container <code>mysql</code> stdout	Query errors, connection errors
Queue worker log	stdout container <code>worker</code>	Job processed/failed

Log	Lokasi	Isi
Cloudflare tunnel log	stdout container app	Tunnel connect/disconnect

7.2.1 Cara Akses Log di Docker

```
# Log Laravel real-time
docker compose logs -f app
# Atau langsung baca file (lebih informatif):
docker compose exec app tail -f storage/logs/laravel.log

# Log Nginx
docker compose logs -f nginx

# Log queue worker
docker compose logs -f worker

# Log scheduler
docker compose logs -f scheduler
```

7.2.2 Log Tailing di Dev (non-Docker)

```
composer run dev
# Sudah include: php artisan pail (log tailing real-time)
```

7.3 Masalah Umum & Solusi

Penjelasan sederhana

Setiap nomor di bawah ini adalah satu jenis masalah. Cocokkan GEJALA yang terlihat dengan judul masalah, lalu ikuti langkah diagnosa dan solusinya.

7.3.1 HTTP 500 — Internal Server Error

Gejala: Halaman menampilkan error 500 (atau halaman putih kosong di production).

Langkah diagnosa:

```
# Cek log terbaru
docker compose exec app tail -20 storage/logs/laravel.log

# Cek apakah .env terkonfigurasi benar
docker compose exec app php artisan env
```

Penyebab umum:

- **APP_KEY** kosong → jalankan `php artisan key:generate`
- **.env** belum di-copy → `cp .env.example .env`
- Permission error di **storage/** → `chmod -R 775 storage bootstrap/cache`
- Database belum migrate → `php artisan migrate`

7.3.2 HTTP 502 Bad Gateway

Gejala: Nginx mengembalikan 502 ketika PHP-FPM tidak bisa diakses.

Penyebab & solusi:

```
# Cek apakah container app berjalan
docker compose ps

# Restart container app
docker compose restart app

# Cek log app
docker compose logs app --tail=50
```

Penyebab umum: container **app** crash saat startup (misal: database belum siap, migration gagal).

7.3.3 Database Connection Failed

Gejala: **SQLSTATE[HY000] [2002] Connection refused** atau **Access denied for user**

Dev (SQLite):

```
# Pastikan file sqlite ada
ls -la database/database.sqlite
# Jika tidak ada:
touch database/database.sqlite
php artisan migrate
```

Dev (MySQL Docker):

```
# Cek container mysql berjalan
docker compose ps mysql

# Cek kredensial di .env cocok dengan docker-compose.yml
grep DB_ .env
grep MYSQL_ docker-compose.yml
```

Production (via Cloudflare Tunnel):

```
# Cek cloudflared berjalan di container
docker compose exec app ps aux | grep cloudflared

# Cek variabel CF_ACCESS_* tersedia
docker compose exec app printenv | grep CF_ACCESS

# Cek log cloudflared
docker compose logs app | grep cloudflared
```

Jika cloudflared gagal koneksi: verifikasi credential di Cloudflare Zero Trust dashboard. Service token mungkin kadaluarsa atau dicabut.

7.3.4 Queue Worker Mati / Job Tidak Diproses

Gejala: Email tidak terkirim, atau job yang seharusnya diproses asinkron tidak berjalan.

Diagnosa:

```
# Cek status container worker
docker compose ps worker

# Cek log worker
docker compose logs worker --tail=50

# Cek tabel jobs di database
docker compose exec app php artisan tinker
>>> DB::table('jobs')->count() # job pending
>>> DB::table('failed_jobs')->count() # job gagal
```

Solusi:

```
# Restart worker
docker compose restart worker

# Retry semua failed jobs
docker compose exec app php artisan queue:retry all

# Jika ada failed job spesifik
docker compose exec app php artisan queue:failed
docker compose exec app php artisan queue:retry <id>

# Flush semua failed jobs (hati-hati — data job hilang)
docker compose exec app php artisan queue:flush
```

7.3.5 Scheduler Tidak Berjalan / Rekap Bulanan Tidak Terjadwal

Gejala: `rekap:bulanan` tidak berjalan otomatis tiap tanggal 1.

Diagnosa:

```
# Cek container scheduler berjalan
docker compose ps scheduler

# Cek log scheduler
docker compose logs scheduler --tail=50

# Verifikasi jadwal terdaftar
docker compose exec app php artisan schedule:list
```

Solusi:

```
# Restart scheduler
docker compose restart scheduler

# Jalankan rekap manual (jika terlambat)
docker compose exec app php artisan rekap:bulanan

# Test schedule tanpa menunggu waktu
docker compose exec app php artisan schedule:run
```

7.3.6 File Upload Tidak Muncul / Storage Link Broken

Gejala: Gambar aset/gedung muncul sebagai broken image, atau file tidak bisa didownload.

Diagnosa:

```
# Cek symlink storage ada
ls -la public/storage
# Harus menunjuk ke: ../storage/app/public

# Cek file ada di volume
docker compose exec app ls storage/app/public/
```

Solusi:

```
# Buat ulang symlink
docker compose exec app php artisan storage:link

# Jika symlink sudah ada tapi broken, hapus dulu:
rm public/storage
docker compose exec app php artisan storage:link
```

Jika di production pakai named volume: pastikan volume `storage_data` ter-mount dengan benar di `docker-compose.prod.yml`.

7.3.7 Assets Tidak Ter-update / CSS Lama

Gejala: Perubahan CSS/JS tidak terefleksikan di browser.

Solusi:

```
# Build ulang assets
npm run build

# Clear cache Laravel
docker compose exec app php artisan view:clear
docker compose exec app php artisan config:clear
docker compose exec app php artisan route:clear

# Hard refresh browser: Ctrl+Shift+R / Cmd+Shift+R
```

Vite memakai hash di nama file — ini berarti browser otomatis load versi baru setelah `npm run build`. Tapi jika cache Laravel menyimpan nama file lama, jalankan `view:clear`.

7.3.8 Halaman Tampil Tapi Data Tidak Berubah / Cache Stale

Gejala: Update data sudah disimpan tapi tampilan tidak berubah.

```
# Clear semua cache
docker compose exec app php artisan cache:clear
docker compose exec app php artisan config:cache
docker compose exec app php artisan route:cache
docker compose exec app php artisan view:cache
```

7.3.9 Akses 403 Forbidden (Tanpa Error)

Gejala: User sudah login tapi mendapat 403 saat mengakses halaman tertentu.

Penyebab: Middleware `CheckMenuAccess` menolak akses karena route tidak ada di daftar menu yang diizinkan untuk role user tersebut.

Diagnosa:

```
# Cek di database: apakah menu dengan route_name tsb ada?
docker compose exec app php artisan tinker
>>> App\Models\Menu::where('route_name', 'nama.route.yang.bermasalah')->first()

# Cek apakah menu.id ada di role user
>>> App\Models\User::find($userId)->role->menu
```

Solusi: Buka halaman admin `/menus` → tambahkan menu entry untuk route tersebut. Buka `/roles` → assign menu ke role yang sesuai.

7.3.10 Error "419 Page Expired" (CSRF Token Mismatch)

Gejala: Form submit gagal dengan "419 Page Expired".

Penyebab: Session kadaluarsa atau CSRF token tidak cocok.

Solusi:

- User: refresh halaman dan submit ulang
- Admin: cek `SESSION_DRIVER=database` dan tabel `sessions` ada (sudah migrate)
- Jika session sering expired: cek `SESSION_LIFETIME` di `.env` (default 120 menit)

7.3.11 Migration Gagal saat Deploy

Gejala: `Illuminate\Database\QueryException: SQLSTATE[42S01]: Base table already exists`

Penyebab: Migration mencoba membuat tabel yang sudah ada (migration sudah di-apply sebelumnya tapi tabel `migrations` tidak mencatat).

Diagnosa:

```
docker compose exec app php artisan migrate:status
```

Solusi (jangan `migrate:fresh!`):

```
# Jika migration sudah applied tapi tidak tercatat:
docker compose exec app php artisan tinker
>>> DB::table('migrations')->insert(['migration' => '2026_01_01_000000_create_tabel_tsb', 'batch' => 1])

# Lalu coba migrate lagi:
docker compose exec app php artisan migrate
```

7.3.12 Cloudflare Tunnel Error — Container Tidak Bisa Koneksi DB

Gejala: `cloudflared: failed to connect` atau timeout koneksi database di production.

Langkah diagnosa:

```
# Cek log cloudflared
docker compose -f docker-compose.prod.yml logs app | grep -i cloudflared

# Cek apakah variabel CF teraset
docker compose -f docker-compose.prod.yml exec app printenv CF_ACCESS_CLIENT_ID
```

Penyebab umum:

1. **Service token kadaluarsa** — buat token baru di Cloudflare dashboard
2. **Hostname salah** — verifikasi `CF_ACCESS_HOSTNAME` di `.env`
3. **Cloudflared binary tidak ada di image** — rebuild image
4. **Cloudflare service down** — cek `status.cloudflarestatus.com`

Sementara jika tunnel tidak bisa dipakai (darurat):

- Setup SSH tunnel manual ke server DB
- Ubah `DB_HOST` ke IP/host yang bisa diakses langsung (jika ada firewall rule yang mengizinkan)

7.4 Checklist Diagnosa Cepat

Penjelasan sederhana

Jika tidak tahu masalahnya apa, kerjakan checklist ini dari atas ke bawah. Kebanyakan masalah sudah ketemu penyebabnya sebelum baris terakhir.

- ☐ Apakah semua container berjalan? → `docker compose ps`
- ☐ Ada error di log Laravel? → `tail storage/logs/laravel.log`
- ☐ Ada error di log Nginx? → `docker compose logs nginx`
- ☐ Database bisa diakses? → `docker compose exec app php artisan tinker >>> DB::select('select 1')`
- ☐ Migration sudah up to date? → `php artisan migrate:status`
- ☐ Storage symlink ada? → `ls -la public/storage`
- ☐ Cache Laravel stale? → `php artisan cache:clear + config:cache`
- ☐ Queue worker berjalan? → `docker compose ps worker`
- ☐ Ada failed jobs? → `php artisan queue:failed`

7.5 Kapan Harus Eskalasi?

Penjelasan sederhana

Eskalasi artinya meneruskan masalah ke orang atau tim yang lebih berwenang/ahli. Masalah selalu dimulai dari Level 1, lalu dinaikkan ke level berikutnya jika tidak selesai dalam batas waktu respons.

Segera eskalasi jika:

- Masalah belum selesai setelah 30 menit mengikuti panduan di bab ini.
- Ada data yang hilang, rusak, atau berubah tanpa sebab yang jelas.
- Ada dugaan masalah keamanan: akun dibobol, spam masif di form publik, atau file mencurigakan terunggah.
- Server database di kantor (on-premise) mati atau Cloudflare Tunnel tidak bisa dipulihkan.
- Perbaikan membutuhkan perubahan kode program atau perubahan langsung ke database.

Informasi yang wajib dikumpulkan sebelum eskalasi:

- Waktu kejadian (tanggal dan jam) serta berapa banyak pengguna yang terdampak.
- Halaman/menu yang bermasalah dan akun yang sedang dipakai.
- Screenshot pesan error yang muncul di layar.
- Langkah yang sudah dicoba beserta hasilnya.
- Isi 50 baris terakhir log aplikasi dan status container (lihat perintah di bawah).
- Apakah ada deploy, perubahan pengaturan, atau pekerjaan server sesaat sebelum masalah muncul.

```
# Ambil 50 baris terakhir log aplikasi
docker compose exec app tail -50 storage/logs/laravel.log
```

```
# Lihat status semua container
docker compose ps
```

Level	Penanggung jawab	Menangani	Waktu respons
Level 1 — Support	Admin SIMASTER / Staf General Affair	Masalah pengguna, reset password, akses menu, error umum	Maks. 1 jam kerja
Level 2 — Developer	Developer / pengelola aplikasi SIMASTER	Error aplikasi, bug, migration, queue, perubahan kode	Maks. 4 jam kerja

Level	Penanggung jawab	Menangani	Waktu respons
Level 3 — Infrastruktur	Tim IT Infrastruktur LM FEB UI	Server, Docker, database on-premise, Cloudflare, backup	Maks. 1 jam (sistem mati total)

Kanal pelaporan: gunakan grup/kanal resmi tim IT LM FEB UI. Untuk kondisi sistem mati total, hubungi Level 2 dan Level 3 sekaligus.

KEPUTUSAN STANDAR & DAFTAR RISIKO

SINGKATNYA

1. Bagian A berisi keputusan standar untuk hal-hal yang jawabannya TIDAK ADA di kode program, misalnya spesifikasi server, jadwal backup, dan CAPTCHA. Setiap poin dilengkapi status: sudah berjalan atau masih harus dikerjakan.
2. Bagian B adalah daftar risiko: hal yang bisa menimbulkan masalah di kemudian hari beserta saran perbaikannya. Jika waktu terbatas, baca tabel "Ringkasan Prioritas" di akhir bab terlebih dahulu. Risiko nomor satu adalah belum adanya backup otomatis.

Bab ini berisi **keputusan standar** untuk hal-hal yang tidak bisa dipastikan dari kode program (Bagian A), dan **risiko teknis** yang perlu diketahui tim operasional (Bagian B).

8.1 Bagian A — Keputusan Standar

Penjelasan sederhana

Bagian ini berisi hal-hal yang jawabannya tidak ada di kode program, sehingga ditetapkan sebagai standar. Kolom "Status di sistem saat ini" menunjukkan mana yang sudah berjalan dan mana yang masih harus dikerjakan oleh developer atau tim infrastruktur.

No	Topik	Keputusan standar	Status di sistem saat ini
A1	Nama institusi	LM FEB UI — General Affair	Selesai
A2	Spesifikasi server	Rekomendasi 4 vCPU / 8 GB RAM / SSD (rincian di Bab 2)	Cek server yang dipakai
A3	Email notifikasi	Diaktifkan untuk notifikasi pengajuan disetujui/ditolak	Perlu disetel di .env
A4	Backup otomatis	Database harian 02:00, file upload mingguan, simpan 30 hari, 2 lokasi	Perlu dipasang di server
A5	Batas ukuran upload	Maksimal 2 MB per file, format JPG/JPEG/PNG/PDF	Perlu diseragamkan di kode
A6	CAPTCHA formulir publik	Wajib, Google reCAPTCHA v3	Perlu ditambahkan ke kode
A7	Retensi audit log	Disimpan 2 tahun di database, lebih lama diarsipkan	Perlu dibuat perintah arsip
A8	Akses per gedung	Tidak dibatasi per gedung; akses diatur lewat role/menu	Selesai (sesuai sistem sekarang)

8.1.1 A1. Nama Institusi

Keputusan: LM FEB UI, unit General Affair. Alamat aplikasi: <https://simaster.lmfebui.com/>.

Tindak lanjut: Tidak ada.

8.1.2 A2. Spesifikasi Server Production

Keputusan: Server aplikasi minimum 2 vCPU / 4 GB RAM / 40 GB SSD, rekomendasi 4 vCPU / 8 GB RAM / 80 GB SSD. Server database minimum 2 vCPU / 4 GB RAM / 50 GB SSD, rekomendasi 4 vCPU / 8 GB RAM / 100 GB SSD + disk backup terpisah. OS Ubuntu Server 24.04 LTS. Rincian lengkap di Bab 2 (subbab Kebutuhan Server).

Tindak lanjut: Tim infrastruktur memastikan server yang dipakai sekarang minimal memenuhi kolom Minimum.

8.1.3 A3. Email Notifikasi

Keputusan: Email **diaktifkan** di production untuk memberi tahu pengaju saat pengajuannya disetujui atau ditolak. Pengirim memakai akun email resmi LM FEB UI (contoh nama pengirim: "SIMASTER LM FEB UI").

Pengaturan: Di `.env` production ubah `MAIL_MAILER=log` menjadi `MAIL_MAILER=smtp`, lalu isi `MAIL_HOST`, `MAIL_PORT` (587, TLS), `MAIL_USERNAME`, `MAIL_PASSWORD`, `MAIL_FROM_ADDRESS`, dan `MAIL_FROM_NAME` sesuai akun email yang disiapkan tim IT. Setelah itu jalankan `php artisan config:cache` dan restart worker (email dikirim lewat antrian).

Tindak lanjut: Tim infrastruktur menyiapkan akun email pengirim; developer memastikan notifikasi terkirim lewat queue.

8.1.4 A4. Backup Otomatis

Keputusan:

- Database: backup **setiap hari pukul 02:00 WIB** (`mysqldump` + gzip).
- File upload (`storage_data`): backup **setiap minggu, hari Minggu pukul 03:00 WIB**.
- Masa simpan: **30 hari** terakhir; ditambah 1 salinan per bulan yang disimpan 12 bulan.
- Lokasi: **2 tempat** — disk backup di server database dan NAS/cloud milik LM FEB UI.
- Uji restore: **1 kali per 3 bulan** ke server uji, untuk memastikan backup benar-benar bisa dipakai.

Tindak lanjut: Tim infrastruktur memasang cron backup (contoh perintah di Bab 6, subbab Backup & Restore).

8.1.5 A5. Batas Ukuran File Upload

Keputusan: Semua formulir yang menerima file memakai aturan yang sama: **maksimal 2 MB per file**, format **JPG, JPEG, PNG, atau PDF**. Batas Nginx tetap 64 MB (untuk formulir dengan beberapa lampiran sekaligus).

Tindak lanjut: Developer menyeragamkan validasi `mimes:jpg,jpeg,png,pdf|max:2048` di semua controller yang menerima upload (terkait risiko R6).

8.1.6 A6. CAPTCHA di Form Publik

Keputusan: **Wajib** memasang **Google reCAPTCHA v3** di semua formulir publik (peminjaman aset/ruangan, permintaan kendaraan, ekspedisi, pengaduan, pengadaan, permintaan barang, pendaftaran vendor). Versi v3 berjalan di belakang layar sehingga pengguna tidak perlu mencentang atau memilih gambar. Batas `throttle:20,1` tetap dipakai.

Tindak lanjut: Developer menambahkan reCAPTCHA (perkiraan 1 hari kerja; terkait risiko R1).

8.1.7 A7. Retensi Data Audit Log

Keputusan: Audit log disimpan **2 tahun** di database. Data yang lebih lama dari 2 tahun **diekspor ke file arsip** (disimpan bersama backup), lalu dihapus dari database. Proses dijalankan otomatis **setiap tanggal 1 pukul 01:00 WIB**.

Tindak lanjut: Developer membuat perintah arsip audit log dan mendaftarkannya ke scheduler (terkait risiko R4).

8.1.8 A8. Akses Multi-Lokasi / Multi-Gedung

Keputusan: **Tidak dibatasi per gedung**. Hak akses cukup diatur lewat role dan menu (RBAC) seperti sekarang. Jika kelak ada kebutuhan membatasi staf hanya ke gedung tertentu, hal itu diajukan sebagai pengembangan fitur baru.

Tindak lanjut: Tidak ada.

8.2 Bagian B — Risk Register

Penjelasan sederhana

Risk register adalah daftar risiko. Setiap risiko dijelaskan: apa risikonya, seberapa mungkin terjadi, seberapa besar dampaknya, dan apa rekomendasi perbaikannya.

8.2.1 R1. Tidak Ada CAPTCHA di Form Publik

Risiko	Bot atau penyerang bisa memenuhi sistem dengan data permintaan palsu
Kemungkinan	Sedang — form publik bisa di-discover, rate limiting bisa di-bypass dengan banyak IP
Dampak	Tinggi — antrian approval penuh dengan data sampah; staff harus memilah manual
Mitigasi saat ini	<code>throttle:20,1</code> per IP — 20 request/menit
Rekomendasi	Tambahkan Google reCAPTCHA v3 (invisible) atau hCaptcha di semua form publik POST
Biaya implementasi	~1 hari kerja — library <code>anhsee/laravel-recaptcha</code> atau integrasi manual

8.2.2 R2. Tidak Ada Backup Otomatis

Risiko	Kehilangan semua data jika server/disk gagal, atau salah jalankan <code>migrate:fresh</code>
Kemungkinan	Rendah per kejadian, tapi konsekuensi permanen
Dampak	Kritis — tidak ada recovery jika database hilang
Mitigasi saat ini	Tidak ada
Rekomendasi	Jadwalkan <code>mysqldump</code> harian via cron di server database; simpan ke minimal 2 lokasi berbeda
Biaya implementasi	~2 jam — setup cron + target backup (disk eksternal / S3 / NAS)

8.2.3 R3. Race Condition pada ID Generator

Risiko	Dua request yang masuk bersamaan bisa mendapat ID yang sama (duplikat PK)
Detail teknis	ID format <code>PREFIX-YYYYMMDD-NNNN</code> di-generate dengan: <code>SELECT MAX(id) LIKE "PJM-YYYYMMDD-%" ORDER BY id DESC LIMIT 1</code> → increment. Tanpa lock, dua request yang masuk dalam hitungan milidetik bisa membaca angka terakhir yang sama.
Kemungkinan	Rendah di kondisi normal (traffic rendah), meningkat saat banyak user submit bersamaan
Dampak	Sedang — gagal insert karena PK conflict (akan ada error 500), tapi data tidak corrupt
Mitigasi saat ini	<code>DB::transaction()</code> + <code>lockForUpdate()</code> saat <code>store()</code> di beberapa modul
Catatan	Perlu audit: tidak semua modul menggunakan <code>lockForUpdate()</code> secara konsisten
Rekomendasi	Audit semua <code>generateId()</code> di controller — pastikan semuanya wrapped dalam <code>transaction</code> + <code>lockForUpdate</code> , atau migrasi ke UUID/auto-increment

8.2.4 R4. Audit Log Tumbuh Tanpa Batas

Risiko	Tabel <code>audit_logs</code> bisa tumbuh sangat besar seiring waktu, memperlambat query
Kemungkinan	Pasti — setiap CRUD di semua model menghasilkan 1+ record audit
Dampak	Sedang — query ke <code>audit_logs</code> menjadi lambat setelah jutaan record; disk penuh
Rekomendasi	Tambahkan scheduled command untuk archive/delete audit log > N bulan; atau partisi tabel berdasarkan bulan
Biaya implementasi	~1 hari kerja

8.2.5 R5. Tidak Ada HTTPS Enforcement

Risiko	Jika diakses via HTTP (bukan HTTPS), session token bisa di-intercept
Status	Bergantung setup Nginx/load balancer di depan — tidak dikonfigurasi di dalam aplikasi

Standar	Aplikasi wajib diakses lewat HTTPS di https://simaster.lmfebui.com . Sertifikat SSL dipasang di depan Nginx (Cloudflare atau reverse proxy kampus); semua akses HTTP dialihkan otomatis ke HTTPS
Rekomendasi	Aktifkan <code>FORCE_HTTPS=true</code> di <code>.env</code> + tambahkan <code>Str::forceScheme('https')</code> di <code>AppServiceProvider</code> jika HTTPS termination ada di luar container

8.2.6 R6. Validasi Upload File — Tipe & Ukuran

Risiko	Penyerang bisa upload file berbahaya (script PHP, malware) menyamar sebagai gambar
Mitigasi saat ini	<code>mimes:jpg,jpeg,png,pdf</code> + <code>max:2048</code> validasi Laravel di beberapa form; <code>intervention/image</code> resize gambar (menghilangkan metadata berbahaya)
Celah	Tidak semua form upload menggunakan validasi yang sama; file PDF tidak di-scan
Rekomendasi	Audit semua form dengan file upload, pastikan semua pakai <code>mimes</code> + <code>max validation</code> secara konsisten

8.2.7 R7. Queue Failure Tanpa Alert

Risiko	Queue worker gagal/mati tanpa ada notifikasi ke admin
Status	Tidak ada alerting
Dampak	Rendah saat ini (queue dipakai untuk job non-kritis), bisa meningkat jika email notifikasi diaktifkan
Rekomendasi	Setup health check container worker + alert (email / Slack) jika container down

8.3 Ringkasan Prioritas

Risk	Prioritas	Effort
R2 — Tidak ada backup	Kritis	Rendah (~2 jam)
R1 — Tidak ada CAPTCHA	Tinggi	Sedang (~1 hari)
R4 — Audit log tanpa batas	Sedang	Sedang (~1 hari)
R3 — Race condition ID	Sedang	Rendah (~2 jam audit)
R5 — HTTPS	Sedang	Rendah (~2 jam: aktifkan redirect HTTPS + <code>forceScheme</code>)
R6 — Upload validation	Sedang	Rendah (~2 jam audit)
R7 — Queue alert	Rendah	Sedang

SOP OPERASIONAL HARIAN

SINGKATNYA

1. SOP (Standard Operating Procedure) adalah langkah-langkah baku untuk pekerjaan rutin, misalnya menambah user, reset password, atau mengatur hak akses.
2. Sebagian besar SOP bisa dikerjakan lewat browser tanpa coding. Langkah yang memakai perintah (kotak abu-abu berhuruf komputer) harus dijalankan oleh orang yang punya akses ke server.

Panduan langkah-per-langkah untuk tugas rutin administrasi sistem. Semua aksi di sini membutuhkan akses superadmin kecuali disebutkan lain.

9.1 SOP-01: Tambah User Baru

Halaman: `/users` → tombol "Tambah User"

Langkah:

1. Login sebagai superadmin
2. Buka menu **Admin** → **Kelola Pengguna** (`/users`)
3. Klik tombol **Tambah User**
4. Isi form:
 - **Nama**: nama lengkap
 - **Email**: email yang akan dipakai untuk login
 - **Password**: sementara (minta user ganti sendiri setelah pertama login)
 - **Role**: pilih role yang sesuai
5. Klik **Simpan**
6. Opsional: jika user perlu akses lebih terbatas dari rolenya, set **menu override** individual (lihat SOP-04)

Catatan: Email harus unik di sistem. Jika email sudah terdaftar akan muncul validasi error.

9.2 SOP-02: Nonaktifkan / Hapus User

Penjelasan sederhana

Disarankan menonaktifkan, bukan menghapus, agar jejak aktivitas user tersebut tetap bisa ditelusuri di audit log.

Perhatian

Perhatian: Hapus user akan menghapus record dari tabel `users`. Audit log yang terkait tetap ada (relasi ke `id_user` bisa menjadi null atau orphan tergantung konfigurasi FK).

Opsi 1 — Nonaktifkan (direkomendasikan): Saat ini belum ada field `is_active` di tabel `users`. Cara sementara: **ganti password** user tersebut ke string random agar tidak bisa login.

```
# Via Artisan Tinker
docker compose exec app php artisan tinker
>>> App\Models\User::find($id)->update(['password' => bcrypt(Str::random(32))])
```

Opsi 2 — Hapus permanen:

1. Buka `/users`
2. Klik ikon hapus di baris user
3. Konfirmasi dialog

Catatan

Standar yang ditetapkan: user yang keluar/pindah **dinonaktifkan, bukan dihapus**. Sampai kolom `is_active` ditambahkan ke tabel `users` oleh developer, gunakan cara Opsi 1 (ganti password acak). Hapus permanen hanya untuk akun yang salah dibuat dan belum pernah dipakai.

9.3 SOP-03: Reset Password User

Via halaman admin:

1. Buka `/users`
2. Klik **Edit** di baris user
3. Isi field **Password Baru + Konfirmasi Password**
4. Klik **Update**

Via Artisan Tinker (jika admin tidak bisa akses UI):

```
docker compose exec app php artisan tinker
>>> App\Models\User::where('email', 'user@example.com')->update(['password' => bcrypt('password_baru')])
```

9.4 SOP-04: Buat Role Baru

Halaman: `/roles` → tombol "Tambah Role"

Langkah:

1. Buka menu **Admin** → **Kelola Role** (/roles)
2. Klik **Tambah Role**
3. Isi **Nama Role**
4. Pilih **menu** yang diizinkan untuk role ini (checkbox list)
5. Klik **Simpan**
6. Assign role ke user: buka /users → Edit user → pilih role baru

9.5 SOP-05: Set Akses Menu per Role (RBAC)

Halaman: /roles

Konsep:

- Setiap role memiliki daftar menu ID yang diizinkan (disimpan sebagai JSON di kolom **menu**)
- User bisa punya **override individual** (kolom **menu** di tabel **users**) yang menimpa role

Ubah akses role:

1. Buka /roles
2. Klik **Edit** di role yang ingin diubah
3. Centang/hapus centang menu yang diizinkan
4. Klik **Simpan**

Override per user individual:

1. Buka /users
2. Klik **Edit** di user
3. Di bagian **Menu Override**: centang menu spesifik
4. Jika kosong (tidak ada yang dicentang) → user mengikuti menu dari rolenya
5. Klik **Update**

Catatan: Jika **menu** di user DAN role keduanya null → user mendapat **full access** (berlaku untuk superadmin).

9.6 SOP-06: Tambah Menu Baru ke Sistem

Penjelasan sederhana

PENTING: halaman yang belum didaftarkan di menu Kelola Menu bisa dibuka oleh SEMUA user yang sudah login. Jadi setiap ada halaman baru, segera daftarkan dan atur siapa yang boleh mengaksesnya.

Dibutuhkan saat: route baru ditambahkan dan perlu dikontrol aksesnya via RBAC.

Halaman: `/menus`

Langkah:

1. Buka **Admin** → **Kelola Menu** (`/menus`)
2. Klik **Tambah Menu**
3. Isi:
 - **Nama:** label yang tampil di sidebar
 - **Route Name:** nama route Laravel (dari `routes/web.php`) — harus sama persis
 - **Icon:** class ikon (contoh: `fas fa-box`)
 - **Parent:** jika sub-menu, pilih menu parent
 - **Urutan:** angka urutan tampil di sidebar
4. Klik **Simpan**
5. Assign menu baru ke role yang seharusnya bisa akses (SOP-05)

Catatan teknis: Jika route tidak ada record di tabel `menus`, middleware `CheckMenuAccess` akan **mengizinkan akses** untuk semua user yang sudah login (bukan 403). Ini desain by default — hanya route yang terdaftar di `menus` yang dikontrol aksesnya.

9.7 SOP-07: Clear Cache (Setelah Deploy / Konfigurasi Berubah)

Jalankan setelah: deploy baru, ubah `.env`, ubah konfigurasi di `config/`.

```
# Via Docker
docker compose exec app php artisan config:clear
docker compose exec app php artisan route:clear
docker compose exec app php artisan view:clear
docker compose exec app php artisan cache:clear

# Lalu rebuild cache untuk performance
docker compose exec app php artisan config:cache
docker compose exec app php artisan route:cache
docker compose exec app php artisan view:cache
```

Perhatian: `config:cache` akan membuat semua `env()` call di luar `config/` tidak berfungsi. Pastikan semua env variable diakses lewat `config()`, bukan `env()` langsung di controller/view.

9.8 SOP-08: Restart Queue Worker

Kapan dibutuhkan: Setelah deploy kode baru (agar worker pakai kode terbaru), atau jika worker mati.

```
# Restart container worker
docker compose restart worker

# Atau sinyal graceful restart (worker selesaikan job berjalan dulu)
docker compose exec app php artisan queue:restart

# Lihat status worker
docker compose logs worker --tail=20
```

Jika ada failed jobs yang perlu di-retry:

```
# Lihat daftar failed jobs
docker compose exec app php artisan queue:failed

# Retry semua
docker compose exec app php artisan queue:retry all

# Retry job tertentu (gunakan ID dari queue:failed)
docker compose exec app php artisan queue:retry 5
```

9.9 SOP-09: Jalankan Rekap Bulanan Gudang Manual

Rekap bulanan gudang berjalan **otomatis** tiap tanggal 1 jam 00:00 via scheduler. Jika perlu dijalankan manual (contoh: scheduler bermasalah, atau rekap bulan lalu terlewat):

```
# Jalankan rekap (menggunakan bulan sebelumnya secara default)
docker compose exec app php artisan rekap:bulanan

# Lihat hasil rekap
# Buka: /gudang/rekap di browser (halaman rekap bulanan)
```

Catatan: Command ini melakukan snapshot `stok_akhir` ke tabel `gudang_rekap_bulanan`. Aman dijalankan berulang — akan update record yang sudah ada (upsert berdasarkan `id_barang` + bulan + tahun).

9.10 SOP-10: Export Data ke Excel/PDF

Tersedia di berbagai halaman daftar (index). Tidak ada langkah khusus — klik tombol Export di UI.

Modul	Format	Tombol di
Aset Inventaris	Excel, PDF	/aset
Peminjaman Aset	PDF	/peminjaman-aset
Gudang Stok	Excel	/gudang
Gudang Rekap	Excel	/gudang/rekap
Laporan Tahunan	Excel, PDF	/laporan-tahunan

File di-generate real-time dan langsung di-download ke browser. File tidak disimpan di server.

9.11 SOP-11: Backup Manual Database

Jalankan sebelum: deploy major, perubahan migrasi besar, atau sebelum maintenance.

```
# Backup dari container MySQL (dev)
docker compose exec mysql mysqldump -u root -p manajemen_aset > backup_$(date +%Y%m%d_%H%M%S).sql

# Backup dari server MySQL eksternal (prod)
# Jalankan dari server database, bukan dari container:
mysqldump -u USERNAME -p DATABASE_NAME > /backups/backup_$(date +%Y%m%d_%H%M%S).sql
```

Simpan file backup di lokasi aman, **bukan di dalam container** (container data hilang saat recreate).

9.12 SOP-12: Proses Darurat — Database Tiba-tiba Tidak Bisa Diakses (Production)

Penjelasan sederhana

Gejalanya: hampir semua halaman error, atau muncul pesan "Connection refused". Ikuti langkah di bawah ini secara berurutan.

Langkah:

1. Cek log container app:

```
docker compose -f docker-compose.prod.yml logs app --tail=50
```

2. Cek cloudflared (jika pakai CF tunnel): Jika cloudflared tidak berjalan: restart container `app`

```
docker compose -f docker-compose.prod.yml exec app ps aux | grep cloudflared
```

```
docker compose -f docker-compose.prod.yml restart app
```

3. Verifikasi credential CF masih valid:

- Login ke Cloudflare Zero Trust dashboard
- Cek service token tidak kadaluarsa
- Jika kadaluarsa: buat token baru, update `.env`, restart `app`

4. Cek server database on-premise berjalan normal (tanya tim IT on-premise)

5. Jika tidak bisa diselesaikan segera: pertimbangkan mode maintenance

```
docker compose -f docker-compose.prod.yml exec app php artisan down --message="Sistem sedang dalam perbaikan. Silakan coba lagi nanti."
# Setelah masalah teratasi:
docker compose -f docker-compose.prod.yml exec app php artisan up
```

9.13 Referensi Cepat

Perintah	Fungsi
php artisan migrate	Jalankan migration baru
php artisan migrate:status	Cek status migration
php artisan rekap:bulanan	Rekap stok gudang manual
php artisan queue:restart	Restart worker (graceful)
php artisan queue:failed	Lihat job gagal
php artisan queue:retry all	Retry semua job gagal
php artisan cache:clear	Hapus cache aplikasi
php artisan config:cache	Rebuild config cache
php artisan route:cache	Rebuild route cache
php artisan view:clear	Hapus compiled views
php artisan down	Mode maintenance
php artisan up	Matikan mode maintenance
php artisan tinker	REPL interaktif untuk debug

LAMPIRAN: GLOSARIUM ISTILAH

Daftar istilah teknis yang dipakai dalam dokumen ini, diurutkan sesuai abjad, beserta arti sederhananya.

Istilah	Arti sederhana
APAR	Alat Pemadam Api Ringan. Salah satu jenis aset yang dikelola sistem.
Aplikasi web	Aplikasi yang dibuka lewat browser (Chrome, Edge, dll.) tanpa perlu diinstal di komputer pengguna.
Artisan	Alat perintah bawaan Laravel untuk menjalankan tugas administrasi, misalnya "php artisan migrate".
Audit log	Catatan otomatis setiap perubahan data: siapa, kapan, dan apa yang diubah. Seperti CCTV untuk data.
Backup / Restore	Backup = membuat salinan cadangan data. Restore = mengembalikan data dari salinan tersebut.
Cache	Simpanan sementara agar aplikasi lebih cepat. Jika tampilan tidak berubah padahal data sudah diubah, cache perlu dibersihkan.
CAPTCHA	Tes "saya bukan robot" untuk mencegah pengisian form otomatis oleh bot.
Cloudflare Zero Trust Tunnel	Jalur koneksi aman dan terenkripsi antara aplikasi dan database di kantor, tanpa membuka database ke internet.
Container	Kotak terisolasi tempat satu bagian aplikasi berjalan. Sistem ini memakai 4-5 container.
Controller	Bagian kode yang mengatur alur: menerima permintaan, mengambil data, lalu menampilkan hasil.
CSRF	Token pengaman di setiap form agar form tidak bisa dikirim dari situs palsu.
Database	Tempat penyimpanan data terstruktur, seperti kumpulan tabel Excel yang saling terhubung.
Deploy	Proses memasang versi terbaru aplikasi ke server asli.
Development / Production	Development = lingkungan uji coba (biasanya di laptop programmer). Production = sistem asli yang dipakai sehari-hari.
Docker	Teknologi untuk membungkus aplikasi dalam container agar berjalan sama di server mana pun.
Endpoint / Route	Alamat URL di aplikasi yang menjalankan fungsi tertentu, misalnya /peminjaman-aset.
.env	File berisi pengaturan rahasia aplikasi (password database, kunci, dll.). Tidak boleh dibagikan.
ERD	Entity Relationship Diagram: peta tabel-tabel database dan hubungannya.
Export	Mengunduh data dari sistem dalam bentuk file Excel atau PDF.
Failed job	Tugas latar belakang yang gagal dijalankan dan menunggu diulang (retry).

Istilah	Arti sederhana
Framework	Kerangka kerja siap pakai untuk membangun aplikasi. Di sini memakai Laravel.
GET / POST / PUT / DELETE	Jenis permintaan ke aplikasi: membuka data / mengirim data baru / mengubah data / menghapus data.
HTTPS / SSL	Koneksi terenkripsi (ikon gembok di browser) agar data tidak bisa disadap.
Laravel	Framework PHP yang dipakai untuk membangun sistem ini.
Library	Komponen kode siap pakai buatan pihak lain, misalnya untuk membuat PDF atau Excel.
Log	Buku harian otomatis sistem yang mencatat kejadian dan error.
Maintenance mode	Mode "sedang perbaikan": pengguna melihat pesan pemberitahuan dan tidak bisa memakai sistem sementara.
Middleware	Penjaga pintu di dalam aplikasi yang memeriksa setiap permintaan, misalnya "sudah login?" dan "punya akses?".
Migration	Resep perubahan struktur database (menambah tabel/kolom) yang dijalankan otomatis.
Model	Bagian kode yang mewakili satu tabel database.
MySQL	Jenis database yang dipakai sistem ini.
Nginx	Web server: pintu depan yang menerima semua kunjungan dari browser.
On-premise	Server yang berada secara fisik di lokasi kantor/institusi sendiri, bukan di cloud.
Opname (stok opname)	Kegiatan menghitung stok fisik barang lalu mencocokkannya dengan catatan sistem.
PHP	Bahasa pemrograman yang dipakai aplikasi ini.
PK / FK	Primary Key = nomor identitas unik setiap baris data. Foreign Key = kolom yang menunjuk ke data di tabel lain.
PNBP	Penerimaan Negara Bukan Pajak, misalnya hasil penjualan/lelang aset yang dimusnahkan.
Queue / Worker	Antrian tugas yang dikerjakan di belakang layar oleh "worker", supaya pengguna tidak perlu menunggu.
Race condition	Masalah ketika dua proses terjadi di waktu yang hampir bersamaan sehingga hasilnya bentrok (misalnya dua transaksi mendapat nomor yang sama).
Rate limiting	Pembatasan jumlah permintaan per menit dari satu alamat internet, untuk mencegah spam.
RBAC	Role-Based Access Control: pengaturan hak akses berdasarkan jabatan (role).
Realisasi	Pemakaian anggaran yang benar-benar terjadi, dibandingkan dengan rencananya.
RKAT	Rencana Kerja dan Anggaran Tahunan.

Istilah	Arti sederhana
Role / Override	Role = jabatan yang menentukan menu apa saja yang boleh dibuka. Override = pengecualian akses khusus untuk satu user.
Scheduler	Penjadwal otomatis yang menjalankan tugas pada waktu tertentu (seperti alarm).
Seeder	Skrip untuk mengisi data awal, misalnya akun superadmin dan daftar menu.
Server	Komputer yang menjalankan aplikasi dan bisa diakses oleh banyak pengguna.
Session	Penanda bahwa seseorang sedang login. Session berakhir setelah waktu tertentu tanpa aktivitas.
SOP	Standard Operating Procedure: langkah baku untuk pekerjaan rutin.
Symlink (storage:link)	Jalan pintas folder agar file upload bisa dibuka lewat browser. Jika hilang, gambar tampil rusak.
Tinker	Konsol interaktif Laravel untuk memeriksa atau mengubah data langsung. Harus dipakai dengan hati-hati.
View / Blade	Bagian kode yang mengatur tampilan halaman.
Volume (Docker)	Tempat penyimpanan permanen milik Docker agar file tidak hilang saat container dibuat ulang.